

**DISTILDOC: DEEP REINFORCEMENT LEARNING FOR
TOKEN-EFFICIENT MULTIMODAL DOCUMENT QUESTION
ANSWERING**

A PROJECT REPORT

Submitted in the partial fulfillment of requirements to

R. V. R & J. C. COLLEGE OF ENGINEERING

**For the award of the degree
B. Tech. in CSE**

By

Patchipulusu Gayathri Asritha (Y22CS144)

Somala Kanth Mani Sai (Y22CS177)

Surampalli Bharat Sai (Y22CS180)



APRIL, 2026

R.V.R. & J. C. COLLEGE OF ENGINEERING (Autonomous)

(Affiliated to Acharya Nagarjuna University)

Chandramoulipuram :: Chowdavaram

GUNTUR – 522019

R.V.R & J.C. COLLEGE OF ENGINEERING
(Autonomous)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE



This is to certify that this project work titled **“DistilDoc: Deep Reinforcement Learning for Token-Efficient Multimodal Document Question Answering”** is the work done by Patchipulusu Gayathri Asritha (Y22CS144), Somala Kanth Mani Sai (Y22CS177), and Surampalli Bharat Sai (Y22CS180) under our supervision, and submitted in partial fulfilment of the requirements for the award of the degree, B. Tech. in Computer Science & Engineering, during the Academic Year **2025-2026**.

Dr. M. Sreelatha
Project Guide

Dr. S J R K Padminivalli V
In-charge, Project Work

Dr. M. Sreelatha
Prof.& Head

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without proper suggestions, guidance and support. Combination of these factors acts like backbone to our project titled “DistilDoc: Deep Reinforcement Learning for Token-Efficient Multimodal Document Question Answering”.

We are very glad to express our special thanks to **Dr. M. Sreelatha**, Project Guide, who has inspired us to select this topic, and also for her valuable suggestion in completion of the project.

We are very thankful to **Dr. S J R K Padminivalli V**, Project In-charge who extended her encouragement and assistance in ensuring the smooth progress of the project work.

We would like to express our profound gratitude to **Dr. M. Sreelatha**, Head of the Department of Computer Science and Engineering for her encouragement and support to carry out this Project successfully.

We are very much thankful to **Dr. Kolla Srinivas**, Principal, for having allowed us to conduct the study needed for the project.

Finally we submit our heartfelt thanks to all the staff in the Department and to all our friends for their cooperation during the project work.

Patchipulusu Gayathri Asritha (Y22CS144)

Somala Kanth Mani Sai (Y22CS177)

Surampalli Bharat Sai (Y22CS180)

ABSTRACT

Document-based question answering using Large Language Models (LLMs) has gained significant importance across domains such as education, healthcare, and scientific research. These systems commonly rely on Retrieval-Augmented Generation (RAG) to provide contextually grounded responses. However, the inclusion of large volumes of retrieved content - especially from multimodal documents containing text, tables, and images - leads to increased token usage, higher inference costs, and reduced efficiency. Existing approaches primarily focus on text-only compression and often employ static or limited adaptive strategies, which are insufficient for handling the complexity and diversity of multimodal information. This project presents a reinforcement learning-based framework for efficient question answering over multimodal documents.

The proposed system processes textual content, structured tables, and visual elements by transforming them into a unified embedding space. A Deep Q-Network (DQN) is employed to model context compression as a sequential decision-making problem, where the agent learns to selectively retain the most relevant information from each modality based on the input query. By dynamically determining compression levels across modalities, the framework ensures that only essential information is passed to the LLM for answer generation. The proposed approach integrates semantic retrieval, multimodal representation, and adaptive compression into a unified pipeline. It is designed to balance answer quality with computational efficiency by reducing unnecessary token usage while preserving critical information. This makes the system suitable for real-world applications involving large and heterogeneous document collections, where both accuracy and cost-effectiveness are essential.

CONTENTS

	Page No.
i Title Page	i
ii Certificate	ii
iii Acknowledgement	iii
iv Abstract	iv
v Contents	v
vi List of Tables	vii
vii List of Figures	viii
viii List of Abbreviations	ix
1. Introduction	1
1.1 Background	1
1.2 Problem Definition	3
1.3 Significance of the work	4
2. Literature Review	
2.1 Review of LLM-based QA Systems	04
2.2 Limitations of existing techniques	08
3. System Analysis	
3.1 Requirements Specification	
3.1.1 Functional Requirements	10
3.1.2 Non-Functional Requirements	11
3.1.3 System Specifications	12
3.2 UML Diagrams	
3.2.1 Use Case Diagram	13
3.2.2 Activity Diagram	15
3.2.3 Sequence Diagram	16

3.2.4 Class Diagram	18
4. System Design	
4.1 Architecture of the proposed system	21
4.2 Workflow of the proposed system	22
4.3 Module Description	23
5. Implementation	
5.1 Algorithms	26
5.2 Data sources Used	43
5.3 Metrics calculated	44
5.4 Methods compared	
6. Testing	47
6.1 Unit Testing	
6.2 Integration Testing	
6.3 System Testing	
6.4 Performance Testing	
6.5 Test Case Validation	
7. Result Analysis	
7.1 Actual Results obtained from the work	50
7.2 Analysis of the Results obtained	50
8. Conclusion and Future Work	52
9. References	53

LIST OF TABLES

S. No	Table No.	Table Description	Page No.
1	6.1	Multimodal Answer Generation Validation	56
2	6.2	Context Compression Efficiency Evaluation	57
3	6.3	Modality-Aware Relevance Adaptation Test	58
4	6.4	Response Latency and Performance Analysis	59
5	6.5	Generalization on Unseen Queries Test	59
6	7.1	Comparative Performance on 200 Samples from ArXiv Dataset	60

LIST OF FIGURES

S. No.	Figure No.	Figure Name	Page No.
1	1.1	Basic Working of Large Language Models	1
2	1.2	RAG Architecture	2
3	2.1	Vector Database Process	6
4	2.2	Cost-Efficient LLM Usage	7
5	2.3	Extractive vs Abstractive Summarization	8
6	2.4	System Overview of LeanContext	11
7	2.5	Deep Q-Network	12
8	2.6	Multimodal Learning	13
9	3.1	Use Case Diagram	21
10	3.2	Activity Diagram	22
11	3.3	Sequence Diagram	24
12	3.4	Class Diagram	26
13	4.1	Multimodal Document Processing	28
14	4.2	DQN Based Context Compression Module	29
15	5.1	Sample Arxiv Research paper Document	51
16	5.2	Sample OpenStax Textbook Page	52
17	7.1	Token Reduction (%) Comparison	
18	7.2	Answer Quality Comparison	
19	7.3	Response Latency Comparison	
20	7.4	Quality vs Efficiency Trade-off	

LIST OF ABBREVIATIONS

1. AI - Artificial Intelligence
2. API - Application Programming Interface
3. Arxiv - Open-access Repository for Scientific Papers
4. BERT - Bidirectional Encoder Representations from Transformers
5. CLIP - Contrastive Language-Image Pretraining
6. DQN - Deep Q-Network
7. GPT - Generative Pre-trained Transformer
8. LLM - Large Language Model
9. ML - Machine Learning
10. NLP - Natural Language Processing
11. QA - Question Answering
12. RAG - Retrieval- Augmented Generation
13. RL - Reinforcement Learning
14. ROUGE - Recall-Oriented Understudy for Gisting Evaluation
15. SBert - Sentence-BERT
16. SimCSE - Simple Contrastive Sentence Embedding
17. T5 - Text- to- Text Transfer Transformer
18. WHO - World Health Organization

1. INTRODUCTION

1.1 Background

In recent years, **Large Language Models (LLMs)** have emerged as a transformative technology in the field of artificial intelligence and natural language processing. LLMs are deep learning models trained on massive amounts of textual data, enabling them to understand, generate, and reason over human language. Popular LLM-based systems are widely used in applications such as chatbots, machine translation, text summarization, and document-based question answering. Their ability to generate contextually relevant and human-like responses makes them highly valuable across domains such as healthcare, education, research, and enterprise systems.

The growing adoption of LLMs is primarily driven by their flexibility and scalability. Instead of building task-specific models, a single LLM can be adapted to multiple applications using prompting techniques. However, LLMs have a significant limitation: they rely only on the knowledge present in their training data and may not have access to recent, domain-specific, or organization-specific information. This limitation becomes critical in applications where up-to-date and accurate information is required.

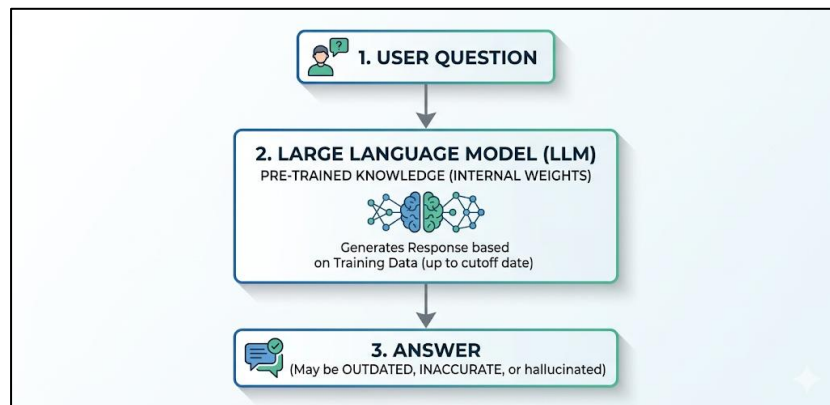


Fig 1.1 Basic working of Large Language Models

To overcome this issue, **Retrieval-Augmented Generation (RAG)** has been introduced as an effective approach for enhancing LLM-based systems. RAG combines information retrieval with text generation by fetching relevant data from external sources and providing it as input context to the LLM. Instead of generating answers

purely based on internal knowledge, the model produces responses grounded in retrieved domain-specific information, thereby improving accuracy and reliability.

In a typical RAG pipeline, documents are first divided into smaller chunks and stored in a vector database using embedding techniques. When a user query is given, the system retrieves the most relevant chunks based on semantic similarity and supplies them as context to the LLM. This enables the model to generate more precise and context-aware answers. As a result, RAG has become a standard approach in modern question-answering systems, including research assistants, knowledge management systems, and educational tools.

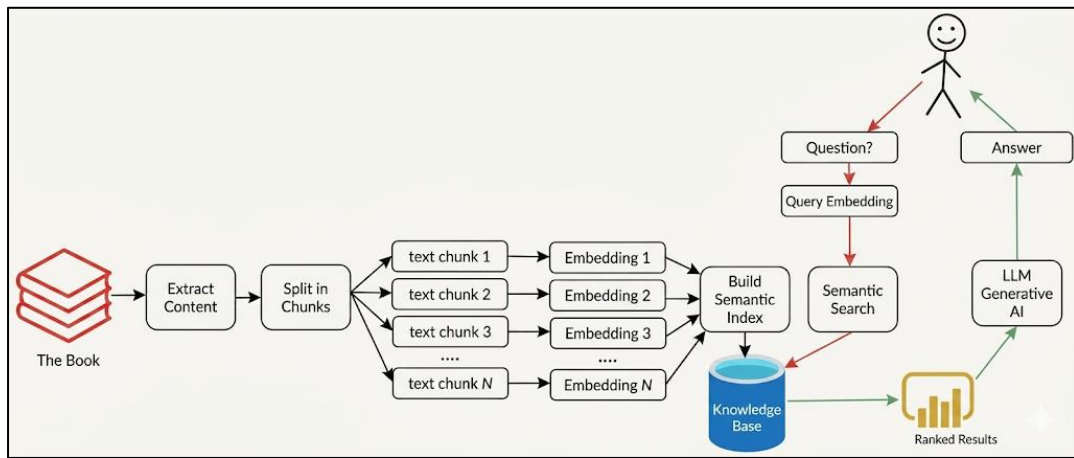


Fig 1.2: Retrieval-Augmented Generation (RAG) architecture

Despite its effectiveness, RAG introduces a critical challenge related to **efficiency and cost**. The retrieval process often returns multiple relevant document segments, all of which are included in the input prompt. As the size of this context increases, the number of tokens processed by the LLM also increases. Since most LLM APIs charge based on token usage, this leads to significantly higher computational cost and slower response times. Additionally, LLMs have a maximum token limit, beyond which they cannot process input effectively, making it difficult to handle large documents.

Another major challenge arises from the nature of real-world documents, which are often **multimodal**. These documents contain not only textual information but also structured tables and visual elements such as charts, graphs, and diagrams. Traditional RAG systems are primarily designed for text-based retrieval and do not effectively utilize information from tables and images. As a result, valuable information present in

non-textual formats may be ignored or underutilized, leading to incomplete or suboptimal answers.

To address these challenges, several approaches have been proposed to reduce the size of the retrieved context using summarization and filtering techniques. These methods aim to retain only the most relevant information before passing it to the LLM. While such approaches improve efficiency to some extent, they are often limited to text-only data and may not adapt well to varying query complexity. Moreover, many existing techniques use fixed or static strategies, which fail to dynamically adjust the amount of context required for different queries.

Therefore, there is a need for an adaptive and efficient context management approach that can handle multimodal data while minimizing token usage without significantly affecting answer quality. This motivates the development of advanced frameworks that combine retrieval, multimodal processing, and intelligent context compression techniques to improve the overall efficiency and scalability of LLM-based question-answering systems.

1.2 Problem Definition

Modern document-based question-answering systems based on Large Language Models (LLMs) face significant challenges when applied to large-scale and real-world datasets. While Retrieval-Augmented Generation (RAG) improves answer accuracy by incorporating external knowledge, it introduces inefficiencies in terms of token usage, computational cost, and context management.

One of the primary issues is the excessive retrieval of context. RAG systems often retrieve multiple document segments for a given query, many of which may contain redundant or less relevant information. Since LLMs process all retrieved content as input, this leads to increased token consumption, higher API costs, and slower response times.

In addition, existing systems struggle with handling multimodal data. Real-world documents often consist of textual content, structured tables, and visual elements such as charts and diagrams. However, traditional RAG-based approaches are primarily designed for text-based retrieval and do not effectively utilize information from tables

and images. This leads to underutilization of valuable information present in non-textual formats.

Another limitation lies in the use of static or non-adaptive context selection strategies. Many existing approaches apply fixed retrieval sizes or generic summarization techniques, which do not consider the complexity or requirements of individual queries. Simple queries may receive excessive context, while complex queries may lack sufficient information, resulting in inefficient or inaccurate responses.

Therefore, there exists a need for an efficient mechanism that can intelligently manage retrieved context, reduce unnecessary token usage, and effectively utilize multimodal information while preserving the quality of generated responses. Addressing these challenges is essential for building scalable and cost-effective document question-answering systems.

1.3 Significance of the work

1. **Efficient Token Utilization:** One of the major challenges in LLM-based systems is the high cost associated with processing large input contexts. This work addresses this issue by reducing unnecessary token usage through intelligent selection of only the most relevant information. This improves computational efficiency while maintaining answer quality.
2. **Multimodal Information Integration:** Unlike traditional approaches that rely primarily on textual data, this work incorporates multiple modalities, including structured tables and visual content such as images and diagrams, into a unified framework. This enables the system to effectively utilize diverse information sources present in real-world documents.
3. **Adaptive Context Compression:** Instead of applying fixed or static compression strategies, the proposed approach dynamically adjusts the amount of context retained based on the specific query. This ensures that simple queries receive minimal context, while complex queries are provided with sufficient relevant information, improving both efficiency and accuracy.
4. **Scalable Learning-Based Approach:** By leveraging a Deep Reinforcement Learning model, specifically a Deep Q-Network (DQN), the system operates on

high-dimensional representations and learns optimal compression strategies. This enables better generalization to unseen queries and enhances scalability compared to traditional rule-based or tabular methods.

5. **Practical Applicability:** The proposed framework is designed with real-world deployment in mind, where cost, latency, and response accuracy are critical factors. It can be effectively applied in domains such as research analysis, educational platforms, and enterprise knowledge management systems.

Overall, this work contributes toward improving the efficiency, scalability, and effectiveness of document-based question-answering systems by introducing a unified, adaptive, and multimodal approach to context selection and compression.

2. LITERATURE REVIEW

2.1 Review of LLM-based QA Systems

The development of document-based question-answering (QA) systems using Large Language Models (LLMs) has gained significant attention in recent years. This section presents a comprehensive review of existing literature across multiple domains, highlighting key contributions and limitations that motivate the proposed work.

2.1.1 Retrieval-Augmented Generation and Vector Databases

Efficient retrieval mechanisms are essential for improving answer accuracy in QA systems. Also, Vector databases play a crucial role in efficient context retrieval by enabling fast and semantic search within large-scale text corpora.

Lewis et al. (2020) [10] – Retrieval-Augmented Generation (RAG)

Lewis et al. (2020) introduce Retrieval-Augmented Generation (RAG), a model architecture that combines pre-trained sequence-to-sequence (seq2seq) models with external nonparametric memory, specifically a dense vector index of Wikipedia. This integration allows RAG to retrieve relevant documents based on a given query and generate informed responses by conditioning on the retrieved content.

Despite its advancements, RAG does not inherently filter or summarize the retrieved documents before generating responses. This means that the model relies on the raw retrieved content, which may include irrelevant or redundant information, potentially affecting the quality and coherence of the generated output.

ChromaDB (2023) [4] – Vector Database

ChromaDB is an open-source vector database designed to efficiently store and retrieve document embeddings, facilitating large language model (LLM)-based retrieval tasks. It supports various features, including embeddings, vector search, document storage, full-text search, metadata filtering, and multi-modal retrieval, all integrated into a single platform.

While ChromaDB significantly improves retrieval efficiency by offering a streamlined approach to managing document embeddings, it does not directly contribute

to the summarization process itself. Its primary function is to enhance the retrieval of relevant documents, which can then be used by LLMs for various tasks.

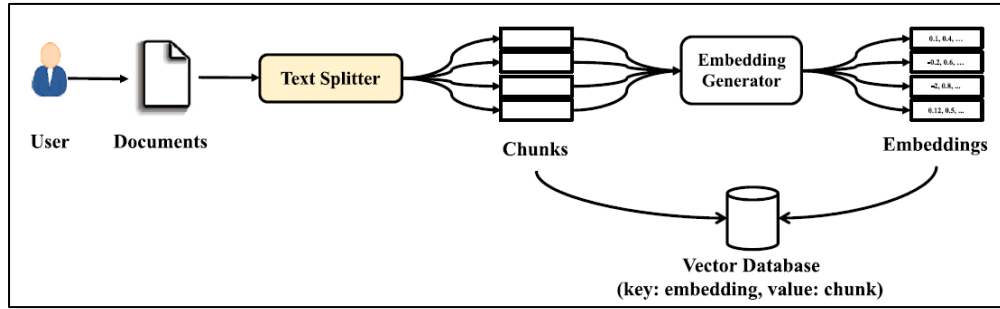


Fig 2.1 Vector Database Process

2.1.2 Cost-Efficient LLM-based Systems

Efficient utilization of LLMs is critical due to the high computational cost associated with token-based processing. Several approaches have been proposed to optimize cost without compromising performance.

Chen et al. (2023) [3] – FrugalGPT: Optimizing Cost and Performance

Chen et al. introduced FrugalGPT, a framework designed to reduce the operational cost of LLM-based applications by dynamically selecting appropriate models based on query complexity. The system intelligently routes simpler queries to smaller, less expensive models while reserving larger models for complex tasks.

However, the approach primarily focuses on model selection rather than reducing the size of the input context. It does not address the issue of excessive token usage caused by large retrieved documents.

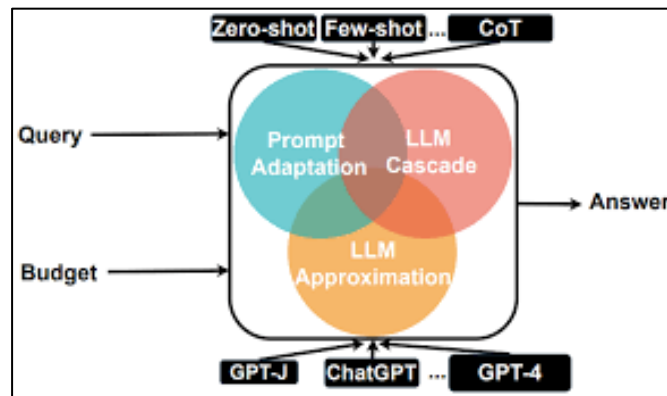


Fig 2.2 Cost-efficient LLM usage

2.1.3 Summarization Techniques

Summarization techniques are essential for reducing document size while preserving important information. These methods are broadly classified into extractive and abstractive approaches.

Miller (2017) [14] – BERT for Extractive Summarization

Miller (2019) introduces a Python-based RESTful service designed to summarize lecture content using BERT embeddings. The system tokenizes lecture transcripts into sentences, generates embeddings via BERT, and applies K-Means clustering to identify central sentences for inclusion in the summary.

While the service effectively identifies central sentences, it does not account for query relevance, limiting its adaptability to specific user needs or contexts. These limitations suggest that incorporating query-aware mechanisms and more flexible extraction strategies could enhance the system's responsiveness and accuracy in generating summaries.

Chung et al. (2022) [5] – T5-Based Summarization

Chung et al. (2022) present Flan-T5, an instruction-tuned variant of the T5 model, fine-tuned on a diverse array of tasks phrased as instructions. This fine-tuning approach enables Flan-T5 to excel in various natural language processing tasks, including abstractive text summarization.

Despite its advancements, the abstractive nature of Flan-T5 introduces challenges related to factual consistency. The model may generate summaries that include hallucinations - information not present in the source text-potentially leading to inaccuracies.

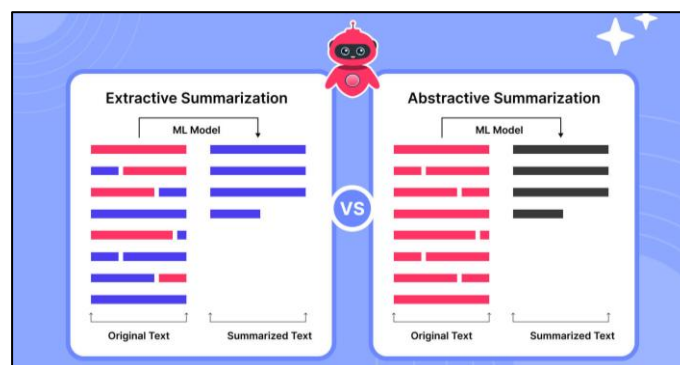


Fig 2.3 Extractive vs Abstractive Summarization

2.1.4 Query-Aware Summarization and Context Optimization

Summarization plays a crucial role in reducing input size while preserving relevant information. However, many traditional approaches are not optimized for query-driven applications.

Laskar et al. (2023) [12] – CQSumDP: Query-Focused Summarization

Laskar et al. proposed CQSumDP, an improved dataset for query-focused summarization, addressing limitations in the existing Debatedpedia dataset. The authors utilized ChatGPT to refine queries and eliminate noise, resulting in more meaningful and contextually relevant summarization tasks.

Despite these improvements, the reliance on ChatGPT introduces potential biases and increases computational overhead. Additionally, the dataset preparation process may not scale efficiently to larger or domain-specific datasets, limiting its broader applicability.

Touvron et al. (2023) [21] – Open-Source Summarization Models

Touvron et al. (2023) present a comprehensive evaluation of open-source large language models (LLMs), including LLaMA 2, for their effectiveness in text summarization tasks. The study benchmarks these models against proprietary counterparts, analyzing their performance in generating concise and coherent summaries across various datasets.

The absence of fine-tuning for specialized fields, such as medical or legal domains, limits the models' effectiveness in generating accurate and contextually relevant summaries for specialized content. Additionally, the study does not address the challenges associated with adapting these models to niche areas where domain-specific knowledge is crucial.

Ling et al. (2023) [13] – Domain-Specialized LLMs

Ling et al. (2023) present a comprehensive survey on techniques for adapting large language models (LLMs) to specialized domains. The paper introduces a systematic taxonomy categorizing various domain-specialization methods based on the accessibility of LLMs, providing frameworks for each subcategory and discussing their interrelations and distinctions.

While the survey extensively covers domain specialization techniques, it does not directly explore their applications in summarization tasks. The focus remains on the methodologies for domain adaptation without assessing how these specialized models perform in generating summaries within specific fields.

2.1.5 Reinforcement Learning-Based Methods

Reinforcement learning (RL) has emerged as a promising technique for optimizing context selection in LLM-based QA systems. RL allows for adaptive decision-making, enabling models to select the most relevant information dynamically based on query requirements. This reduces unnecessary token usage while ensuring accurate responses.

Yang et al. (2023) [23] – RL-Summarizer

Yang et al. (2023) introduce the RL-Summarizer, a model that employs reinforcement learning (RL) to dynamically assess and optimize the importance of sentences within a text for summarization purposes. By utilizing RL, the model learns a policy that rewards the selection of sentences contributing most effectively to the informativeness and coherence of the generated summary. This approach allows the summarizer to adaptively prioritize content.

However, it is trained on general datasets and lacks domain-specific knowledge, which reduces its effectiveness in specialized applications.

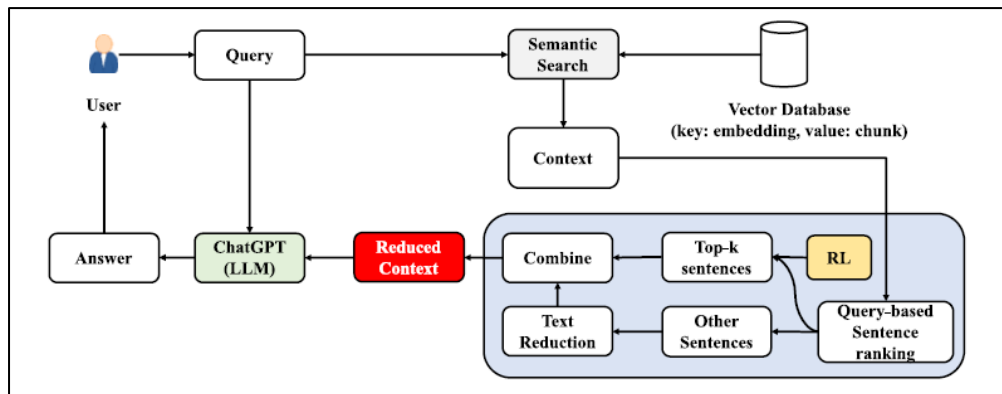


Fig 2.4 System Overview of LeanContext

Arefeen et al. (2024) [1] - LeanContext

LeanContext is a novel framework designed to address the challenges of high token usage and inefficient context selection in LLM-based domain-specific question-

answering systems. LeanContext selectively extracts only the most relevant sentences required for answering a query. This is achieved through a reinforcement learning-based mechanism that dynamically determines the optimal number of sentences to retain.

However, LeanContext primarily focuses on textual data and does not fully address the challenges associated with multimodal documents, such as images and tables. The framework also relies on reinforcement learning, which may introduce additional training complexity and computational overhead.

Mnih et al. (2015) [15] – Deep Q-Network (DQN)

Mnih et al. introduced the Deep Q-Network (DQN), a ground-breaking reinforcement learning approach that combines Q-learning with deep neural networks to enable agents to learn optimal decision-making policies directly from high-dimensional input spaces. The model utilizes techniques such as experience replay and target networks to stabilize training and improve convergence.

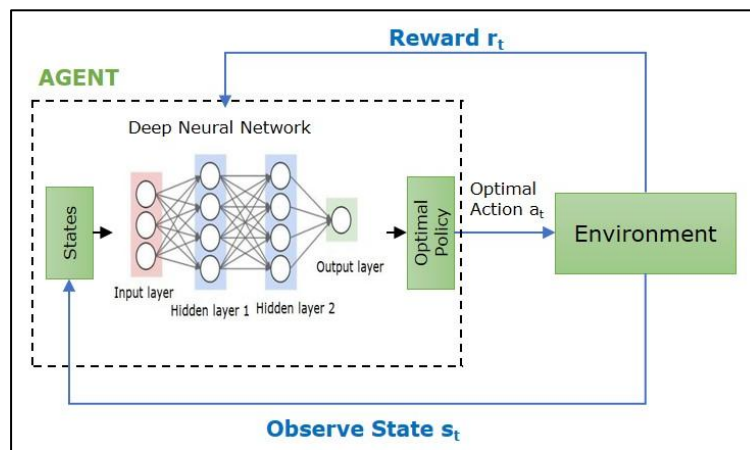


Fig 2.5 Deep Q Network

However, the application of DQN is primarily limited to control-based environments and does not directly address challenges in natural language processing or document-based question-answering systems. The model operates on structured state-action representations and does not inherently support textual or multimodal data.

2.1.6 Semantic Compression Techniques

Semantic compression aims to reduce input size while preserving meaning, which is essential for efficient LLM usage. Researchers have explored various techniques to

achieve semantic compression, ranging from rule-based methods to deep learning-based approaches.

Gilbert et al. (2023) [8] – Semantic Compression with LLMs

Gilbert et al. (2023) investigate the potential of large language models (LLMs), specifically GPT-3.5 and GPT-4, to perform semantic compression on text and code. The study explores how these models can generate compressed representations that maintain the semantic integrity of the original content.

But the problem is by focusing on general semantic preservation without tailoring to particular informational needs, the compressed representations might lack essential nuances required for accurate and comprehensive responses in domain-specific applications.

2.1.7 Multimodal Representation Learning

Multimodal systems are essential for handling real-world documents, which often contain a combination of text, images, tables, and graphical elements. Traditional natural language processing approaches focus primarily on textual data and fail to effectively utilize information present in visual formats.

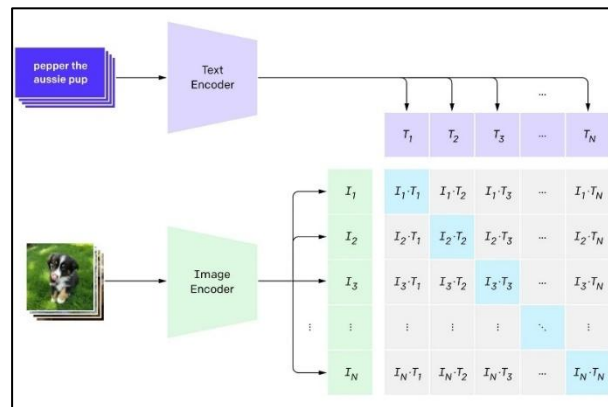


Fig 2.6 Multimodal Learning

Radford et al. (2021) [18] – CLIP

Radford et al. introduced CLIP (Contrastive Language–Image Pretraining), a multimodal model that learns joint representations of images and text by training on large-scale image-caption datasets. The model employs a contrastive learning approach,

where it aligns images and their corresponding textual descriptions in a shared embedding space.

However, despite its strong representation capabilities, CLIP is not specifically designed for integration into question-answering pipelines. It does not provide mechanisms for query-based retrieval, context filtering, or adaptive compression of multimodal information.

2.2 Limitations of Existing Techniques

Despite significant advancements in document-based question-answering systems and Retrieval-Augmented Generation (RAG) frameworks, several limitations remain, especially when handling large-scale and multimodal data. Traditional RAG systems rely on static retrieval strategies, selecting a fixed number of document chunks regardless of query complexity. This often leads to either insufficient context for complex queries or redundant information for simpler ones, resulting in inefficiency.

Most existing retrieval and summarization techniques are designed primarily for text-based data and fail to effectively utilize information from tables and images, thereby overlooking important multimodal content. Additionally, while dense retrieval methods improve relevance, they do not reduce context size, leading to increased token usage, higher computational cost, and latency. Many summarization approaches generate static or query-independent summaries, which may either omit critical information or include unnecessary details.

Although techniques such as semantic compression and reinforcement learning improve efficiency and adaptability, they introduce additional computational overhead and often lack support for multimodal data. Furthermore, existing methods do not provide a unified framework that integrates adaptive context selection, multimodal processing, and token-efficient inference.

To overcome these limitations, the proposed work focuses on developing an adaptive and multimodal-aware framework that dynamically selects relevant information while minimizing token usage, thereby improving both efficiency and answer quality.

3. SYSTEM ANALYSIS

A Software Requirements Specification (SRS) is a document that describes what the system will do and how it is expected to perform. It defines both functional and non-functional requirements to ensure the successful development and deployment of the system.

3.1 Requirements Specification

Requirement analysis, also known as requirement engineering, is the process of identifying and defining the needs and expectations of users for a software system. These requirements must be clear, measurable, and aligned with the objectives of the system. In this project, the system is designed to perform efficient document-based question answering using multimodal data while minimizing token usage and maintaining answer quality.

3.1.1 Functional Requirements

A Functional Requirement (FR) describes the specific functionalities that the system must perform. The functional requirements for the system are:

1. Query Processing:

The system should accept user queries in natural language format and preprocess them for analysis. It should convert queries into embeddings for semantic understanding.

- Accepts input query from user interface
- Performs pre-processing such as tokenization and cleaning
- Generates vector embeddings for similarity matching

2. Multimodal Document Processing:

The system should process documents containing text, tables, and images, and convert them into a unified representation for further processing.

- Extracts textual content from documents
- Identifies and processes tables and structured data
- Generates descriptions or features from images
- Converts all modalities into a common embedding format

3. Vector Storage and Retrieval:

The system should store processed document data in a vector database and efficiently retrieve relevant content based on query similarity.

- Stores embeddings in a vector database
- Performs similarity search using query embeddings
- Retrieves top relevant document segments

4. Adaptive Context Selection:

The system should dynamically select relevant content based on the query instead of using a fixed retrieval size.

- Adjusts context size based on query complexity
- Selects only the most relevant information
- Avoids redundant or irrelevant data

5. Context Compression:

The system should reduce the size of retrieved context while preserving important information required for answering queries.

- Filters out less relevant content
- Removes redundant information
- Retains key sentences and important data

6. Deep Reinforcement Learning-Based Decision Making:

The system should use a deep reinforcement learning model to determine optimal strategies for context selection and compression.

- Uses models such as Deep Q-Network (DQN)
- Learns optimal actions based on query and context
- Improves performance through training and feedback

7. Answer Generation:

The system should generate context-aware responses using a language model.

- Uses LLM to generate answers
- Utilizes compressed context as input

- Ensures relevance and correctness of output

8. Multimodal Integration:

The system should combine information from multiple modalities to generate a coherent response.

- Integrates text, table, and image information
- Aligns different data representations
- Produces unified and meaningful output

9. Evaluation Capability:

The system should evaluate the quality and efficiency of generated responses.

- Measures relevance and accuracy of answers
- Evaluates token usage and efficiency
- Supports performance monitoring and improvement

3.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance constraints of the system. In this project, each requirement is associated with measurable criteria to ensure proper evaluation.

1. Performance:

The system should generate responses within a reasonable time, ensuring low latency during query processing and answer generation.

- Measures total response time from query input to answer output
- Evaluates latency under different query complexities
- Target: Response generated within a few seconds

2. Efficiency:

The system should minimize token usage while maintaining answer quality to reduce computational cost.

- Compares token usage before and after context compression
- Evaluates reduction in input size
- Measures cost efficiency in terms of tokens processed

3. Scalability:

The system should handle increasing document sizes and query loads without significant degradation in performance.

- Tests retrieval performance with large document collections
- Evaluates response time with increasing number of documents
- Observes system behaviour under multiple queries

4. Reliability:

The system should produce consistent and accurate responses across different queries and document types.

- Evaluates correctness of generated answers
- Checks consistency for similar queries
- Measures response quality using evaluation metrics (ROUGE-L, Semantic Similarity)

5. Usability:

The system should provide a simple and user-friendly interface for interaction.

- Ensures easy input of queries
- Provides clear and understandable outputs
- Evaluates user interaction experience

6. Security and Privacy:

The system should ensure safe handling of input data and prevent unauthorized access.

- Avoids storing sensitive user data
- Ensures secure API usage
- Maintains data confidentiality during processing

3.1.3 System Specification

Hardware Requirements

- **CPU:** A modern multi-core processor (minimum dual-core; quad-core recommended for better performance).

- **GPU:** Recommended for training deep learning models (CUDA-enabled NVIDIA GPU preferred).
- **Memory:** Minimum 8 GB RAM (16 GB recommended for handling large documents).
- **Storage:** Minimum 20 GB free storage for datasets, models, and embeddings.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Programming Language: Python
- Libraries and Frameworks: PyTorch, NumPy, Pandas, Django
- NLP and Embedding Tools: Cohere Embeddings / similar embedding models
- Vector Database: ChromaDB / Pinecone
- APIs: LLM APIs (e.g., Groq / Gemini / OpenAI-compatible APIs)
- Platforms: Google Colab / Jupyter Notebook / VS Code

3.2 UML Diagrams

UML is an acronym that stands for Unified Modeling Language. Simply put UML is a modern approach to modeling and documenting software. In fact, it is one of the most popular business process modeling techniques.

It is based on diagrammatic representations of software components. Any complex system is best understood by making some kind of diagrams or pictures. These diagrams have a better impact on our understanding. If we look around, we will realize that the diagrams are not a new concept but it is used widely in different forms in different industries.

The various UML diagrams are :

1. Use case diagram
2. Activity diagram
3. Sequence diagram
4. Collaboration diagram
5. Object diagram
6. Class diagram
7. Component diagram
8. Deployment diagram

3.2.1 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor.

Identification of Use-Cases and Sub Use-Case

Use-cases diagrams graphically represent system behaviour (use cases). These diagrams present a high-level view of how the system is used as viewed from an actor's perspective. A use-case diagram may contain all or some of the use cases of a system.

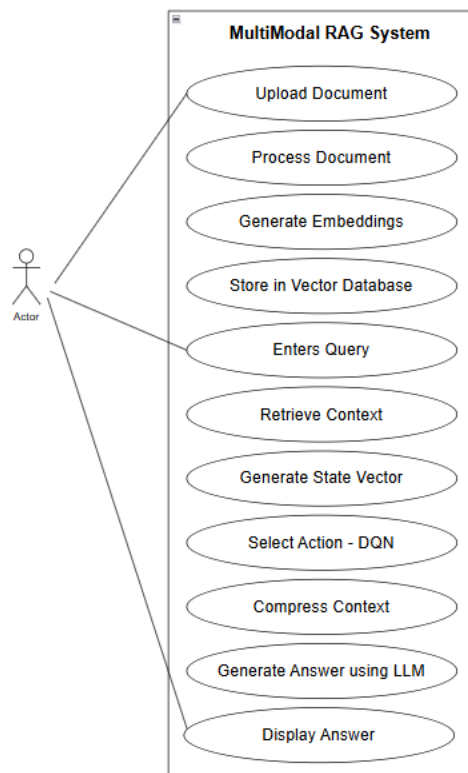


Fig 3.1 Use case Diagram

3.2.2 Activity Diagram

An activity diagram is a behavioral UML diagram that represents the workflow of a system. It illustrates the sequence of activities, decision points, and the overall flow of control from the start to the end of a process. Activity diagrams are useful for modelling

the dynamic aspects of a system and understanding how different components interact during execution.

In the proposed system, the activity diagram represents the complete workflow of the multimodal document processing and query answering pipeline. It includes both document ingestion and query processing phases, along with decision-making using the Deep Q-Network for adaptive context compression.

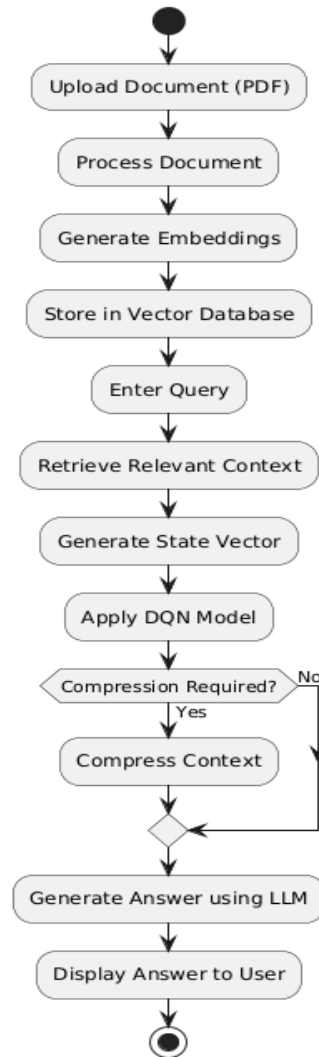


Fig 3.2 Activity Diagram

Process:

1. User uploads a document (PDF).
2. The system processes the document:
 - Extracts text, tables, and images
 - Converts them into structured formats

3. The system generates embeddings for all extracted content.
4. Embeddings are stored in the vector database.
5. User enters a query.
6. The system retrieves relevant context from the database.
7. The system generates a state vector based on retrieved data.
8. The DQN model selects an action using the state vector.
9. The system compresses the retrieved context based on the selected action.
10. The compressed context is passed to the language model.
11. The system generates the final answer.
12. The answer is displayed to the user.

3.2.3 Sequence Diagram

A sequence diagram is a behavioural UML diagram that represents how objects interact with each other in a time-ordered sequence. It shows the flow of messages exchanged between different components of the system to accomplish a specific task.

In the proposed system, the sequence diagram illustrates the interaction between the user, document processing module, embedding model, vector database, Deep Q-Network (DQN), and the language model during document ingestion and query processing. It highlights how data flows through the system and how the final response is generated.

This sequence diagram illustrates the following steps:

1. The user uploads a document to the system.
2. The system processes the document by extracting text, tables, and images.
3. The embedding model generates vector representations for all extracted content.
4. The system stores embeddings in the vector database.
5. The user submits a query to the system.
6. The system sends the query to the embedding model to generate a query vector.
7. The vector database retrieves relevant context based on similarity.
8. The system constructs a state vector using retrieved embeddings.
9. The state vector is passed to the DQN model.
10. The DQN model selects an optimal compression action.
11. The system compresses the retrieved context based on the selected action.

12. The compressed context and query are sent to the language model.
13. The language model generates the final answer.
14. The system returns the generated answer to the user.

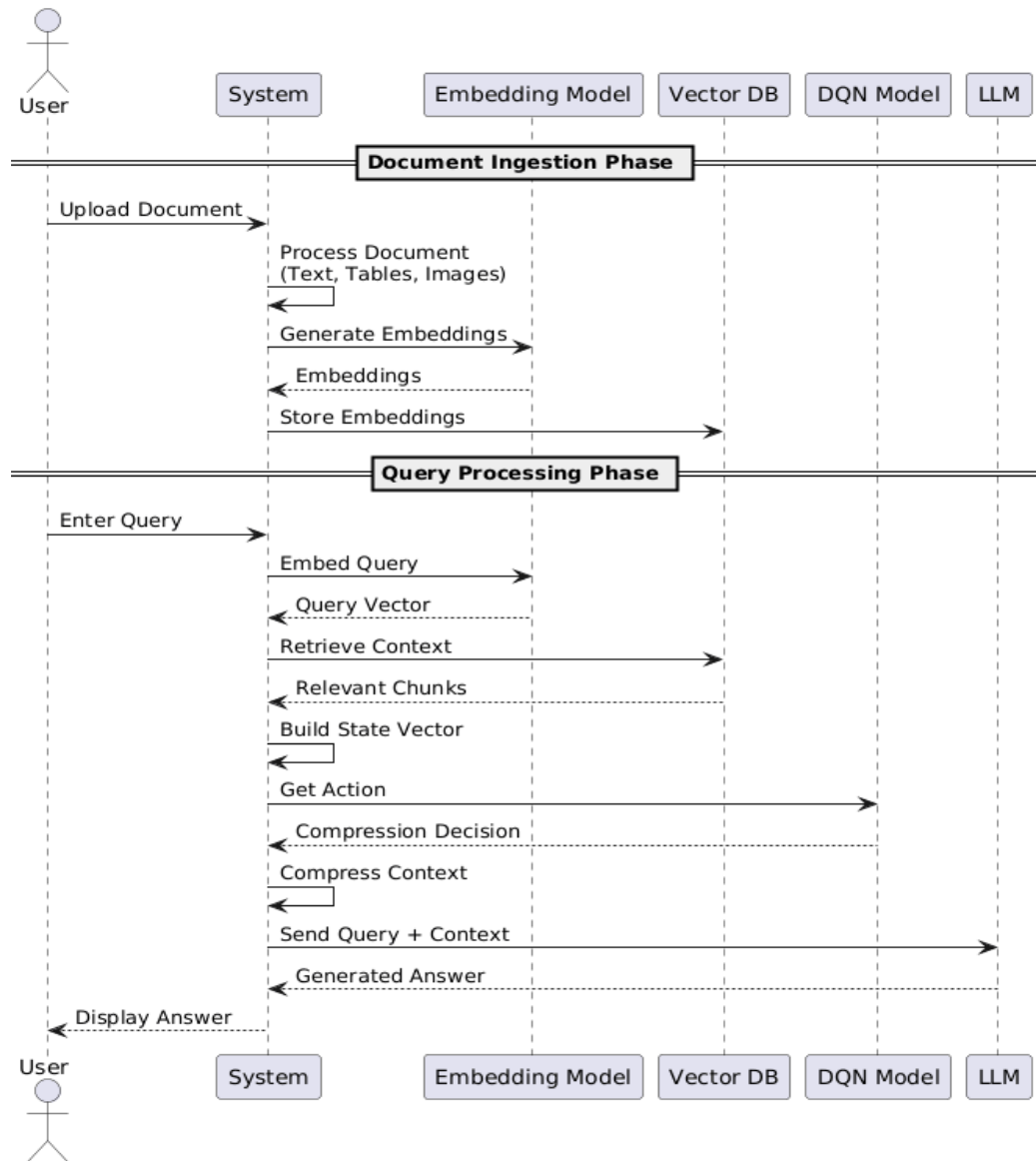


Fig 3.3 Sequence Diagram

3.2.4 Class Diagram

A class diagram is a structural UML diagram that represents the static structure of a system by showing its classes, attributes, methods, and relationships among them. It helps in understanding how different components are organised and interact with each other.

In the proposed system, the class diagram represents key components involved in multimodal document processing, embedding generation, retrieval, reinforcement learning, context compression, and answer generation.

Identification of Attributes and Methods of Class

Classes:

- **Document:** This class represents the uploaded document. It is responsible for extracting and storing text, tables, and image captions from the document.
- **Embedding Model:** This class generates vector embeddings for different modalities such as text, tables, and images.
- **Vector Database:** This class stores embeddings and retrieves relevant context based on query similarity.
- **DQN Model:** This class represents the Deep Q-Network that selects optimal compression actions using the state vector.
- **Context Compressor:** This class applies compression on retrieved context using thresholds selected by the DQN model.
- **LLM Model:** This class generates the final answer based on the compressed context and user query.

Attributes:

- Document: document_id, raw_content, text_chunks, table_rows, img_captions
- Embedding Model: embedding_dimension, model_name
- Vector Database: stored_embeddings, metadata
- DQN Model: input_size, hidden_layers, output_size, weights
- Context Compressor: text_threshold, table_threshold, image_threshold
- LLM Model: model_name, temperature

Operations:

- Document: split_document(), extract_text(), extract_tables(), extract_images()
- Embedding Model: generate_embedding(input_data)
- Vector Database: store_embedding(), retrieve_context(query_vector)
- DQN Model: predict_action(state_vector)
- Context Compressor: compress_context(context, action)
- LLM Model: generate_answer(query, context)

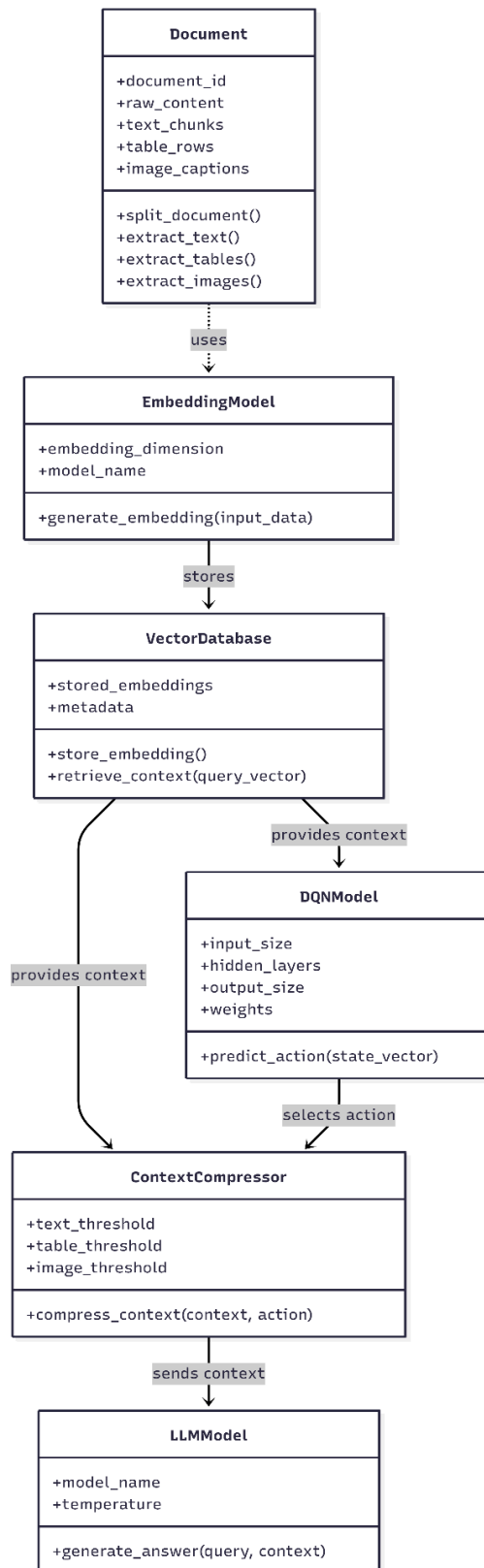


Fig 3.4 Class Diagram

4. SYSTEM DESIGN

4.1. Architecture of the proposed system

The proposed system is a multimodal document question-answering framework designed to efficiently reduce the amount of context sent to a Large Language Model (LLM) while preserving answer quality. The architecture is organized into two major functional components:

- (i) Multimodal Document Processing and Embedding Pipeline, and
- (ii) Deep Reinforcement Learning-based Context Compression Module.

These components are utilized across both the training phase (offline learning) and the inference phase (online execution) of the system.

(i) Multimodal Document Processing and Embedding Pipeline:

This component is responsible for processing input documents and converting them into a unified embedding space. It handles multiple modalities including text, tables, and images by transforming them into vector representations. These embeddings are stored in a vector database and are used during both training and inference for semantic retrieval.

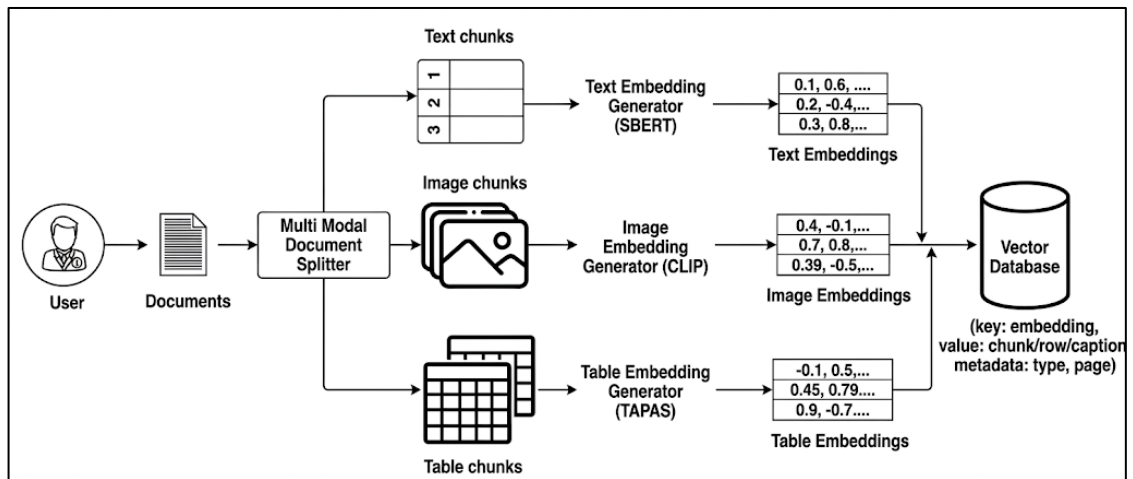


Fig 4.1 Multimodal Document Processing

This component is active in both training and inference phases for document processing and context retrieval.

(ii) DQN-based Context Compression Module:

This component is responsible for adaptive context selection and compression using a Deep Q-Network (DQN). It takes the retrieved multimodal context as input and determines the optimal amount of information to retain based on the query.

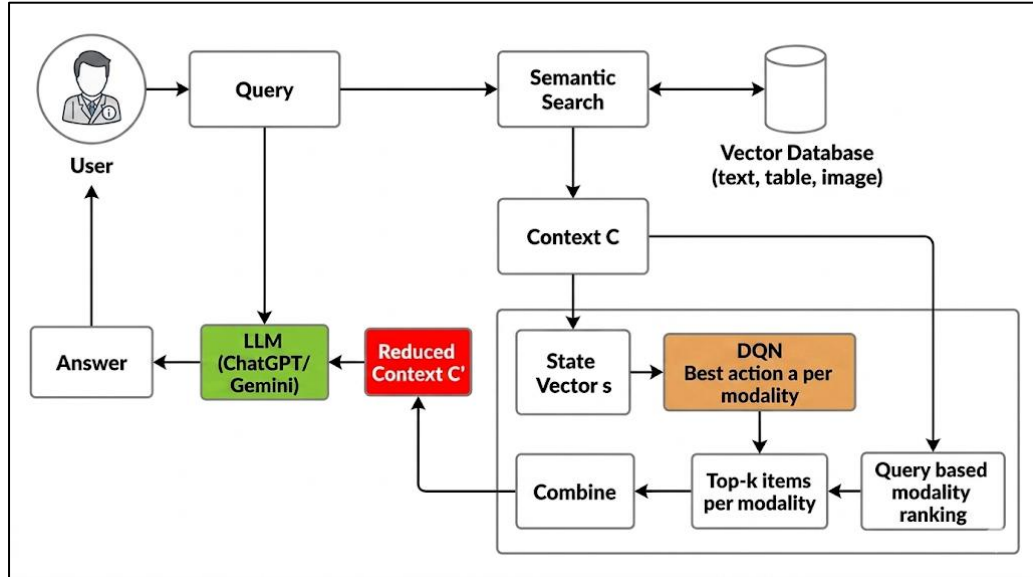


Fig 4.2 DQN based Context Compression Module

During the training phase, this component learns optimal compression strategies using rewards. During the inference phase, it uses the trained model to make decisions without further learning.

4.2 Workflow of proposed system

The workflow of the proposed system is divided into two major phases: Training Phase (Offline Learning) and Inference Phase (Online Execution). Both phases utilize the architectural components shown in Fig 4.1.1 and Fig 4.1.2, but differ in functionality.

Phase 1: Training Phase (Offline Learning)

In this phase, the system learns optimal context compression strategies using reinforcement learning. Both the multimodal processing pipeline (Fig 4.1.1) and the DQN module (Fig 4.1.2) are actively involved.

The steps involved in the Training Phase of DQN are as follows:

1. **Document Ingestion and Pre-processing:** The input document is first divided into three modalities: text, tables, and images. Text is split into manageable chunks, tables are converted into row-wise textual representations, and images are converted into captions using a vision-language model.
2. **Embedding and Storage:** All processed elements are transformed into dense vector embeddings using a shared embedding model. These embeddings are stored in a vector database along with metadata such as modality type, page number, and token count.
3. **Query Processing and Context Retrieval:** For each training query, the system generates a query embedding and retrieves the most relevant items from each modality using semantic similarity search. These retrieved elements form the initial context.
4. **State Representation Construction:** The system computes the mean embedding for each modality and subtracts the query embedding to form modality-specific alignment vectors. These vectors are concatenated to form a unified state vector representing the relationship between the query and the multimodal context.
5. **Action Selection using DQN:** The state vector is passed through the Deep Q-Network, which outputs Q-values for all possible compression actions. An action is selected using an exploration-exploitation strategy.
6. **Context Compression:** Based on the selected action, the system filters and retains only a fraction of the most relevant content from each modality, producing a reduced context.
7. **Answer Generation and Reward Computation:** The reduced context is passed to the LLM to generate an answer. This answer is compared with a reference answer generated from the full context to compute a reward based on semantic similarity, lexical overlap, and token efficiency.
8. **Model Update:** The experience consisting of state, action, and reward is stored and used to update the DQN parameters, allowing the model to learn better compression strategies over time.

Phase 2: Inference Phase (Online Execution)

In this phase, the trained system generates answers for new queries without further learning. The same components (Fig 4.1.1 and Fig 4.1.2) are used, but only for execution.

1. **Query Input and Embedding:** The user submits a query, which is converted into a dense embedding.
2. **Context Retrieval:** Relevant text chunks, table rows, and image captions are retrieved from the vector database using semantic similarity.
3. **State Vector Formation:** The system constructs the state vector using the same formulation as in training, capturing the relationship between the query and retrieved content.
4. **DQN-Based Decision Making:** The trained DQN performs a forward pass and selects the optimal compression action without any further training.
5. **Context Compression:** The retrieved context is filtered according to the selected action, retaining only the most relevant information across modalities.
6. **Answer Generation:** The compressed context is passed to the LLM, which generates the final answer.
7. **Response Output:** The generated answer is returned to the user with reduced latency and lower computational cost.

4.3 Module Description

The proposed system is composed of several interconnected modules that work together to perform efficient multimodal document question answering. Each module is responsible for a specific stage from document ingestion to final answer generation.

a) Multimodal Document Processing Module

This module is responsible for converting raw input documents into structured components that can be processed by the system. Since real-world documents contain heterogeneous content, the system separates the document into three modalities: text, tables, and images.

- Text content is divided into fixed-size chunks to preserve semantic meaning.
- Tables are transformed into row-wise natural language representations so that structured data can be processed similarly to text.
- Images are converted into descriptive captions using a vision-language model.

This modular separation ensures that each type of information is preserved and can be effectively utilized during retrieval and compression.

b) Embedding and Vector Storage Module

Once the document is processed, this module converts all elements into dense vector representations using a shared embedding model. These embeddings enable semantic comparison between queries and document content.

- Each text chunk, table row, and image caption is converted into a high-dimensional embedding vector.
- All embeddings are stored in a vector database along with metadata such as modality type, page number, and token count.

This unified embedding space allows efficient similarity-based retrieval across all modalities.

c) Semantic Retrieval Module

This module is responsible for retrieving relevant information from the vector database based on the user query.

- The input query is converted into an embedding vector.
- A similarity search is performed to retrieve most relevant items from each modality.
- The retrieved results forms the initial multimodal context.

This module ensures that only semantically relevant information is passed to subsequent stages, reducing unnecessary data processing.

d) DQN-based Compression Module

This is the core module of the system, responsible for reducing the size of the retrieved context while preserving essential information.

- A state representation is constructed by comparing the query embedding with modality-specific embeddings.
- The state is passed to a Deep Q-Network (DQN), which selects the optimal compression strategy.
- The selected action determines how much content to retain from each modality.

This module enables dynamic, query-aware compression, allowing the system to adapt to different types of questions and document structures.

e) Answer Generation Module

This module generates the final answer using the compressed context.

- The reduced context is passed to a Large Language Model.
- The model generates a response based only on the relevant information retained after compression.

By limiting the input size while preserving important content, this module ensures both efficiency and high-quality responses.

These modules operate together in a unified pipeline, enabling efficient multimodal retrieval, adaptive compression, and accurate answer generation while significantly reducing token usage.

5. IMPLEMENTATION

5.1 Algorithms

The proposed system utilises a Deep Q-Network (DQN)-based reinforcement learning approach to learn and apply optimal context compression strategies for multimodal Retrieval-Augmented Generation. The overall implementation is divided into two primary algorithms:

1. DQN Based Training Algorithm - where the DQN learns effective compression policies
2. DQN Based Inference Algorithm - where the trained model is used to generate efficient and high-quality responses.

1. DQN-Based Training Algorithm

The training phase is conducted offline and aims to teach the DQN how to select the most effective compression strategy for different queries and multimodal contexts.

a) Multimodal Data Preparation

Initially, all input documents are processed and divided into three modalities:

- Text → split into manageable chunks
- Tables → converted into row-wise representations
- Images → converted into captions using a vision model

Each of these components is transformed into dense vector embeddings using a pretrained embedding model. These embeddings are stored in a vector database along with metadata such as modality type and source information. This enables efficient retrieval of relevant content during training.

b) Context Retrieval

For each training query, the system retrieves the most relevant items from the vector database using semantic similarity (cosine similarity).

The retrieved set of items from all three modalities forms the initial context (C), which serves as the input for further processing.

c) State Representation

To allow the DQN to make decisions, the retrieved context is converted into a numerical state representation.

First, the embeddings of retrieved items in each modality are averaged to obtain: v_{text} , v_{table} , v_{image}

These are then compared with the query embedding v_q . The difference between them represents how well each modality aligns with the query.

The final state vector is defined as:

$$s = [v_{\text{text}} - v_q, v_{\text{table}} - v_q, v_{\text{image}} - v_q]$$

This combined vector captures the relevance of all three modalities in a single representation and is used as input to the DQN.

d) Action Selection (ϵ -Greedy Policy)

The DQN selects an action based on the current state using an ϵ -greedy strategy:

- With probability ϵ , a random action is chosen (exploration)
- Otherwise, the best-known action is selected (exploitation)

Each action represents a compression strategy, defined as a combination of retention ratios for: Text, Tables and Images. Each action is represented as a three-tuple:

$$a = (a_{\text{text}}, a_{\text{table}}, a_{\text{image}})$$

For example, an action may specify retaining 20% of text, 30% of tables, and 10% of images.

e) Context Compression

Once action is selected, initial context C is compressed to produce reduced context C' .

This ensures that only the most relevant information is preserved.

- Items in each modality are ranked based on similarity to the query
- Only the top-ranked items, as defined by the selected compression ratios, are retained
- Remaining items are discarded

f) Answer Generation

The compressed context (C') is passed to the language model along with the query to generate a response:

- Reduced Answer (A_{red}) $\rightarrow$ generated using compressed context
- Full Answer (A_{full}) $\rightarrow$ obtained from full context (precomputed or cached)

These two answers are used to evaluate the quality of compression.

g) Reward Computation

A reward is computed to measure how well the compressed context preserves answer quality while reducing token usage.

$$R = \alpha * \text{Sim}(A_{red}, A_{full}) + \beta * \text{ROUGE-L}(A_{red}, A_{full}) - \gamma * \tau$$

Where:

- Semantic similarity evaluates meaning preservation
- ROUGE evaluates lexical overlap
- Token ratio (τ) penalises larger outputs
- The weights α , β and γ determine the relative importance of the different objectives within the reward function.

This reward guides the learning process of the DQN.

h) Experience Replay

Each interaction is stored as a tuple: (s,a,r)

These experiences are saved in a replay buffer. During training:

- Random mini-batches are sampled
- This reduces correlation between consecutive samples
- Improves stability and generalisation of learning

i) Network Training and Optimisation

The DQN is updated using the sampled experiences from the replay buffer.

- The network predicts the usefulness of each action for a given state
- Predictions are compared with target values

- The difference is reduced using gradient-based optimisation

A separate target network is used to provide learning signals and is updated periodically.

Over multiple episodes, the DQN learns to associate query-context patterns with optimal compression strategies.

Implementation code:

DistilDoc: DQN Training Pipeline

```
import os
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import Adam
from rouge_score import rouge_scorer
from sklearn.metrics.pairwise import cosine_similarity
import cohere
from groq import Groq
```

CONSTANTS - Action Space and Hyperparameters

```
TEXT_THRESHOLDS = [0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40] # 8 values
TABLE_THRESHOLDS = [0.10, 0.20, 0.30, 0.40] # 4 values
IMAGE_THRESHOLDS = [0.10, 0.20, 0.30, 0.40] # 4 values

# All 128 action tuples: (a_text, a_table, a_image)
ACTIONS = [
    (at, ab, ai) for at in TEXT_THRESHOLDS for ab in TABLE_THRESHOLDS
    for ai in IMAGE_THRESHOLDS ]
NUM_ACTIONS = len(ACTIONS) # 128
EMBED_DIM = 1536
STATE_DIM = 3 * EMBED_DIM # 4608
```

```

# Reward function coefficients ( $\alpha=0.7$ ,  $\beta=0.2$ ,  $\gamma=0.1$ )
ALPHA = 0.7 # weight for semantic similarity
BETA = 0.2 # weight for ROUGE-L score
GAMMA = 0.1 # weight for token compression penalty

# Training hyperparameters
LEARNING_RATE = 1e-4 # Adam optimiser step size
EPSILON_START = 1.0 # initial exploration rate
EPSILON_MIN = 0.05 # minimum exploration rate
EPSILON_DECAY = 0.95 # multiplicative decay per episode
REPLAY_BUFFER_CAP = 500 # maximum transitions stored
MINI_BATCH_SIZE = 16 # transitions sampled per update
TARGET_SYNC_STEPS = 5 # gradient steps between  $\theta^- \leftarrow \theta$  sync

# DQN ARCHITECTURE
# Input : 4608-dimensional state vector s
# Output : 128 Q-values, one per (a_text, a_table, a_image) action tuple
class DQN(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(STATE_DIM, 512) # input  $\rightarrow$  hidden layer 1
        self.fc2 = nn.Linear(512, 256) # hidden layer 1  $\rightarrow$  hidden layer 2
        self.out = nn.Linear(256, NUM_ACTIONS) # hidden layer 2  $\rightarrow$  Q-values

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = F.relu(self.fc1(x)) # ReLU activation on first hidden layer
        x = F.relu(self.fc2(x)) # ReLU activation on second hidden layer
        return self.out(x) # linear output - no activation

# REPLAY BUFFER
# Stores transitions (s, a, r) and enables random mini-batch sampling.

```

```

class ReplayBuffer:
    def __init__(self, capacity: int = REPLAY_BUFFER_CAP):
        self.buf = []
        self.capacity = capacity

    def push(self, state: torch.Tensor, action: int, reward: float):
        if len(self.buf) >= self.capacity:
            self.buf.pop(0)
        # Store state as list for JSON serialisation compatibility
        self.buf.append((state.tolist(), int(action), float(reward)))

    def sample(self, n: int) -> list:
        """Sample n random transitions and reconstruct tensors."""
        batch = random.sample(self.buf, min(n, len(self.buf)))
        return [(torch.FloatTensor(s), a, r) for s, a, r in batch]

# EMBEDDING UTILITY

def embed(text: str, co: cohere.ClientV2, input_type: str = "search_query"):
    res = co.embed( model = "embed-v4.0", input_type = input_type,
                    embedding_types = ["float"], texts = [text] )
    return np.array(res.embeddings.float[0])

# STATE VECTOR CONSTRUCTION

def compute_state(query_emb: np.ndarray, text_embs: list[np.ndarray],
                  table_embs: list[np.ndarray], image_embs: list[np.ndarray]):
    def mean_emb(embs: list[np.ndarray]) -> np.ndarray:
        if not embs:
            return np.zeros(EMBED_DIM) # zero-fill missing modalities
        return np.mean(embs, axis=0)
    v_text = mean_emb(text_embs)
    v_table = mean_emb(table_embs)
    v_image = mean_emb(image_embs)

```

```

state = np.concatenate([ v_text - q_emb, v_table - q_emb, v_image - q_emb ])
return torch.FloatTensor(state)

```

ACTION SELECTION

```

def select_action(state: torch.Tensor, policy: DQN, epsilon: float):
    if random.random() < epsilon:
        action_idx = random.randint(0, NUM_ACTIONS - 1) # explore
    else:
        policy.eval()
        with torch.no_grad():
            action_idx = int(policy(state).argmax().item()) # exploit

    a_text, a_table, a_image = ACTIONS[action_idx]
    return action_idx, a_text, a_table, a_image

```

CONTEXT COMPRESSION

```

def compress_context(context: dict, action: tuple, query_emb: np.ndarray):
    a_text, a_table, a_image = action
    compressed = {}
    for modality, threshold in [("text", a_text), ("table", a_table), ("image", a_image)]:
        items = context.get(modality, [])
        if not items:
            compressed[modality] = []
            continue
        embs = np.array([item["embedding"] for item in items])
        k = max(1, ceil(threshold * len(items)))
        top_k_indices = np.argsort(sims)[-k:][::-1]
        compressed[modality] = [items[i] for i in sorted(top_k_indices)]
    return compressed

```

REWARD COMPUTATION

```

def compute_reward(answer_red: str, answer_full: str, tokens_red: int, tokens_full: int,

```

```

        co: cohere.ClientV2) -> float:

# Semantic similarity via cosine distance between Cohere embeddings
e_red = embed(answer_red[:300], co).reshape(1, -1)
e_full = embed(answer_full[:300], co).reshape(1, -1)
sem_sim = float(cosine_similarity(e_red, e_full)[0][0])

# ROUGE-L: lexical overlap via longest common subsequence
scorer = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)
rouge_l = scorer.score(answer_full, answer_red)["rougeL"].fmeasure

# Token ratio  $\tau$  = compressed tokens / full tokens
tau = tokens_red / max(tokens_full, 1)

return float(ALPHA * sem_sim + BETA * rouge_l - GAMMA * tau)

# DQN UPDATE

# Samples mini-batch from B, computes TD loss, updates  $\theta$  via Adam.
def update_dqn(policy: DQN, target: DQN, optimizer: torch.optim.Optimizer,
               buffer: ReplayBuffer, step_counter: list) -> float | None:
    if len(buffer) < MINI_BATCH_SIZE:
        return None # wait until buffer has enough transitions

    # Sample random mini-batch — breaks temporal correlation
    batch = buffer.sample(MINI_BATCH_SIZE)
    states = torch.stack([b[0] for b in batch]) # (16, 4608)
    actions = torch.tensor([b[1] for b in batch], dtype=torch.long) # (16,)
    rewards = torch.tensor([b[2] for b in batch], dtype=torch.float32) # (16,)

    # Predicted Q-values for the actions that were actually taken
    q_pred = policy(states).gather(1, actions.unsqueeze(1)).squeeze(1) # (16,)

    # TD target:  $y = r$  (single-step episode — no next-state bootstrap needed)
    with torch.no_grad():

```

```

    q_target = rewards # y = r
    loss = F.mse_loss(q_pred, q_target)
    optimizer.zero_grad()
    loss.backward()
    # Gradient clipping prevents exploding gradients from noisy LLM rewards
    torch.nn.utils.clip_grad_norm_(policy.parameters(), 1.0)
    optimizer.step()
    step_counter[0] += 1
    if step_counter[0] % TARGET_SYNC_STEPS == 0:
        target.load_state_dict(policy.state_dict()) #  $\theta^- \leftarrow \theta$ 

    return loss.item()

```

LLM ANSWER GENERATION

```

def generate_answer(query: str, context: dict, groq_client: Groq) -> tuple[str, int]:
    # Concatenate all modality texts into a single context string
    context_text = "\n".join( item["text"] for modality in ["text", "table", "image"]
                               for item in context.get(modality, []))
    prompt = ( f"Answer based only on the context provided.\n\n"
               f"Context:\n{context_text}\n\nQuestion: {query}" )
    response = groq_client.chat.completions.create(
        model = "llama-3.1-8b-instant", messages = [{"role": "user", "content":
            prompt}], temperature = 0.3, max_tokens = 512 )
    answer = response.choices[0].message.content.strip()
    token_count = response.usage.prompt_tokens
    return answer, token_count

```

MAIN TRAINING LOOP

```

def train(documents: list[dict], queries: list[dict], episodes_per_query: int = 5,
save_path: str = "distildoc_dqn.pt"):
    co = cohere.ClientV2(api_key=os.getenv("COHERE_API_KEY"))
    groq_client = Groq(api_key=os.getenv("GROQ_API_KEY"))

```



```

buffer    = ReplayBuffer(capacity=REPLAY_BUFFER_CAP) # line 1: init B
policy    = DQN()
target    = DQN()
target.load_state_dict(policy.state_dict())          #  $\theta^- \leftarrow \theta$ 
optimizer = Adam(policy.parameters(), lr=LEARNING_RATE)
epsilon    = EPSILON_START
step_counter = [0] # mutable counter for target sync tracking

for q_idx, query_dict in enumerate(queries):
    question = query_dict["question"]
    ref_answer = query_dict.get("answer", "")
    for episode in range(1, episodes_per_query + 1):
        context_full = documents[q_idx % len(documents)]
        query_emb = embed(question, co, input_type="search_query")
        text_embs = [item["embedding"] for item in context_full.get("text", [])]
        table_embs = [item["embedding"] for item in context_full.get("table", [])]
        image_embs = [item["embedding"] for item in context_full.get("image", [])]

        state = compute_state(query_emb, text_embs, table_embs, image_embs)
        action_idx, a_text, a_table, a_image = select_action( state, policy, epsilon)
        context_compressed = compress_context(
            context_full, (a_text, a_table, a_image), query_emb )
        answer_red, tokens_red = generate_answer(question, context, client)

        if ref_answer:
            answer_full = ref_answer
            tokens_full = sum( len(item["text"].split()) for m in ["text", "table", "image"]
                               for item in context_full.get(m, []) )
        else:
            answer_full, tokens_full = generate_answer(question, context, groq_client)
        reward =
        compute_reward( answer_red, answer_full, tokens_red, tokens_full, co)

```

```

buffer.push(state, action_idx, reward)
loss = update_dqn(policy, target, optimizer, buffer, step_counter)
epsilon = max(EPSILON_MIN, epsilon * EPSILON_DECAY)
torch.save(
    {"policy": policy.state_dict(), "target": target.state_dict()}, save_path )
print(f"\n[DistilDoc] Training complete. Checkpoint saved: {save_path}")
return policy

```

2. DQN-Based Inference Algorithm

After training, the system enters the inference phase, where the learned policy is applied to new queries. This phase is efficient and does not involve any learning or reward computation.

a) Query Processing and Retrieval

When a user submits a query:

- The query is converted into an embedding
- Relevant multimodal content is retrieved from the vector database
- This forms the initial context (C)

b) State Construction

The system constructs the state vector using the same method as in training:

$$s = [v_{text} - v_q, v_{table} - v_q, v_{image} - v_q]$$

This ensures consistency between training and inference.

c) Action Selection (Greedy Policy)

The trained DQN processes the state vector and selects the optimal action:

- The action with the highest predicted value is chosen
- No exploration is performed during inference

This action determines how much information to retain from each modality.

d) Context Compression

The retrieved context is compressed based on the selected action:

- Items are ranked using similarity scores
- Top-k items per modality are retained
- Remaining items are removed

This produces the reduced context (C').

e) Final Answer Generation

The compressed context (C') is passed to the language model along with the query to generate the final answer.

This approach ensures:

- Reduced token usage
- Faster response time
- Maintained answer quality

f) Efficiency of the Inference Process

The inference pipeline is highly efficient as it requires only:

- One embedding computation
- One DQN forward pass
- One LLM call

Since no training or reward computation is involved, the system achieves low latency while maintaining high performance.

Implementation Code for Inference:

```
import torch
```

```
import numpy as np
```

Deep Q-Network (DQN) model for selecting optimal context compression action

```
class DQN(nn.Module):
```

```
    def __init__(self, input_dim=4608, output_dim=128):
```

```
        super(DQN, self).__init__()
```

```
        # Fully connected neural network with ReLU activations
```

```
        self.net = nn.Sequential( nn.Linear(input_dim, 512), nn.ReLU(),
                                   nn.Linear(512, 256), nn.ReLU(),
```

```
nn.Linear(256, output_dim) )
```

```
def forward(self, x):  
    # Forward pass to compute Q-values for all actions  
    return self.net(x)
```

Load trained DQN model for inference

```
def load_dqn_model(model_path):  
    model = DQN()  
    model.load_state_dict(torch.load(model_path, map_location=torch.device('cpu')))  
    model.eval() # Set model to evaluation mode  
    return model
```

Define discrete action space (compression thresholds for each modality)

```
def build_action_space():  
    text_thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]  
    table_thresholds = [0.1, 0.2, 0.3, 0.4]  
    image_thresholds = [0.1, 0.2, 0.3, 0.4]  
    actions = []  
    # Generate all combinations of thresholds (total = 128 actions)  
    for t in text_thresholds:  
        for tb in table_thresholds:  
            for im in image_thresholds:  
                actions.append((t, tb, im))  
    return actions
```

Construct state vector representing query-context relationship

```
def build_state(v_text, v_table, v_image, v_query):  
    # Compute alignment between query and each modality  
    s_text = v_text - v_query  
    s_table = v_table - v_query  
    s_image = v_image - v_query
```

```

# Concatenate all modality vectors to form final state
state = np.concatenate([s_text, s_table, s_image])
return torch.tensor(state, dtype=torch.float32).unsqueeze(0)

# Compress retrieved context based on selected action thresholds
def compress_context(context, action):
    text_thresh, table_thresh, image_thresh = action
    # Select top-k items based on relevance score
    def filter_items(items, threshold):
        if len(items) == 0:
            return []
        k = max(1, int(len(items) * threshold)) # Ensure at least one item
        return sorted(items, key=lambda x: x["score"], reverse=True)[:k]

    # Apply filtering for each modality
    compressed = {
        "text": filter_items(context["text"], text_thresh),
        "table": filter_items(context["table"], table_thresh),
        "image": filter_items(context["image"], image_thresh),
    }
    return compressed

# Format compressed context into a single string for LLM input
def format_context(C_prime):
    text_part = "\n".join([x["content"] for x in C_prime["text"]])
    table_part = "\n".join([x["content"] for x in C_prime["table"]])
    image_part = "\n".join([x["content"] for x in C_prime["image"]])
    return f"""" TEXT: {text_part} TABLE: {table_part} IMAGE: {image_part}""""

# Retrieve relevant multimodal context
def retrieve_context(query):

```

```

return {
    "text": [{"content": "text chunk", "score": np.random.rand()} for _ in range(4)],
    "table": [{"content": "table row", "score": np.random.rand()} for _ in range(3)],
    "image": [{"content": "image caption", "score": np.random.rand()} for _ in
range(2)],
}

```

Compute mean embeddings for each modality

```

def compute_mean_embeddings(context):
    def mean_embed(items):
        if len(items) == 0:
            return np.zeros(1536)
        vectors = [embed(x["content"]) for x in items]
        return np.mean(vectors, axis=0)
    return ( mean_embed(context["text"]), mean_embed(context["table"]),
            mean_embed(context["image"]) )

```

Call LLM to generate final answer

```

def call_llm(query, context_text):
    return f"Generated answer for: {query}"

```

Main inference pipeline

```

def run_inference(query, model, actions):
    # Step 1: Retrieve multimodal context
    context = retrieve_context(query)

    # Step 2: Generate query embedding
    v_q = embed(query)

    # Step 3: Compute mean embeddings for each modality
    v_text, v_table, v_image = compute_mean_embeddings(context)

    # Step 4: Build state representation
    state = build_state(v_text, v_table, v_image, v_q)

```

Step 5: Select optimal action using trained DQN

```
with torch.no_grad():  
    q_values = model(state)  
    action_idx = torch.argmax(q_values).item()  
    action = actions[action_idx]
```

Step 6: Compress context based on selected action

```
C_prime = compress_context(context, action)
```

Step 7: Format compressed context

```
formatted_context = format_context(C_prime)
```

Step 8: Generate answer using LLM

```
answer = call_llm(query, formatted_context)  
return answer, action
```

5.2 Data sources used

The effectiveness of the proposed system is evaluated using multiple document collections that represent diverse real-world multimodal data. Since the system is designed to operate on heterogeneous inputs consisting of text, tables, and images, it is essential to use datasets that reflect this diversity. Therefore, three distinct document corpora are selected, each capturing different characteristics of multimodal content and varying levels of structural complexity.

1. Arxiv Research Papers:

The first dataset consists of research papers obtained from publicly available repositories such as arXiv. These documents contain dense technical content, including textual explanations, structured tables, and illustrative figures.

Fields in Dataset:

- Text content (paragraphs and sections)
- Tables (experimental results, comparisons)
- Images (figures, diagrams)
- Metadata (title, authors, section headings)

a performance drop in the ‘oracle’ setup compared to the original T5. This is because it must infer the answer from a compact summarized representation of the page, while T5 has access to the whole page representation. This shows that the page representation obtained by the encoder has still margin for improvement.

Finally, identifying the page where the answer is found at the same time as answering the question allows to better interpret the method’s results. In Tab. 2 we can see that Hi-VT5 obtains a better answer page identification performance than all the other baseline methods. In addition, in Fig. 5 we show that it is capable to predict the correct page even when it cannot provide the correct answer. Interestingly, it answers correctly some questions for which the predicted page is wrong, which means that the answer has been inferred from a prior learned bias instead of the actual input data. We provide more details by analyzing the attention of Hi-VT5 in Appendix F.

	Answer		
	anls = 1	$0.5 \leq \text{anls} < 1$	anls < 0.5
Answer page prediction Correct	1613	1286	1098
Wrong	242	284	496

Figure 5. Matrix showing the Hi-VT5 correct and wrong answered questions depending on the answer page prediction module result.

6. Ablation studies

To validate the effectiveness of each feature proposed in Hi-VT5, we perform an ablation study and show re-

other works in the literature [2, 9, 18], the most relevant information is conveyed within the text and its position, while explicit visual features are not specially useful for grayscale documents.

Method	Accuracy	ANLS	Ans. Page Acc.
Hi-VT5	48.28	0.6201	79.23
–2D-pos	46.12	0.5891	78.21
–Vis. Feat.	46.82	0.5999	78.22
–APPM	47.78	0.6130	00.00
–Pretrain	42.10	0.5864	81.47
–Fine-tune	42.86	0.6263	55.74

Table 3. **Hi-VT5 ablation studies.** We study the effect of removing different components independently from Hi-VT5 namely the 2D position embedding (2D-pos), visual features (Vis. Feat.), the answer page prediction module (APPM), the pretraining (Pretrain) and the last fine-tuning (Fine-tune) phase of the decoder and answer page prediction module.

7. Conclusions

In this work, we propose the task of Visual Question Answering on multi-page documents and make public the MP-DocVQA dataset. To show the challenges the task poses to current DocVQA methods, we convey an analysis of state-of-the-art methods showing that even the ones designed to accept long sequences are not capable to answer questions posed on the final pages of a document. In order to address these limitations, we propose the new method Hi-VT5 that, without increasing the model complexity, can accept sequences up to 20,480 tokens and answer the questions regardless of the page in which the answer is placed. Finally, we show the effectiveness of each of the components in the method, and perform an analysis of the results showing how the answer page prediction module can help to identify answers that might be inferred from prior learned bias instead of the actual input data.

Fig 5.1 Sample Arxiv Research Paper Document

Sample Data:

- Text: “Proposed model achieves ROUGE score of 0.78 on benchmark dataset.”
- Table: Model | ROUGE Score → LeanContext | 0.78
- Image: “Architecture diagram of the proposed model”

This dataset is useful for evaluating the system’s ability to process complex and highly interdependent multimodal information.

2. OpenStax Textbooks:

The second dataset is derived from the OpenStax collection of academic textbooks. These documents provide structured and well-organized content across multiple domains.

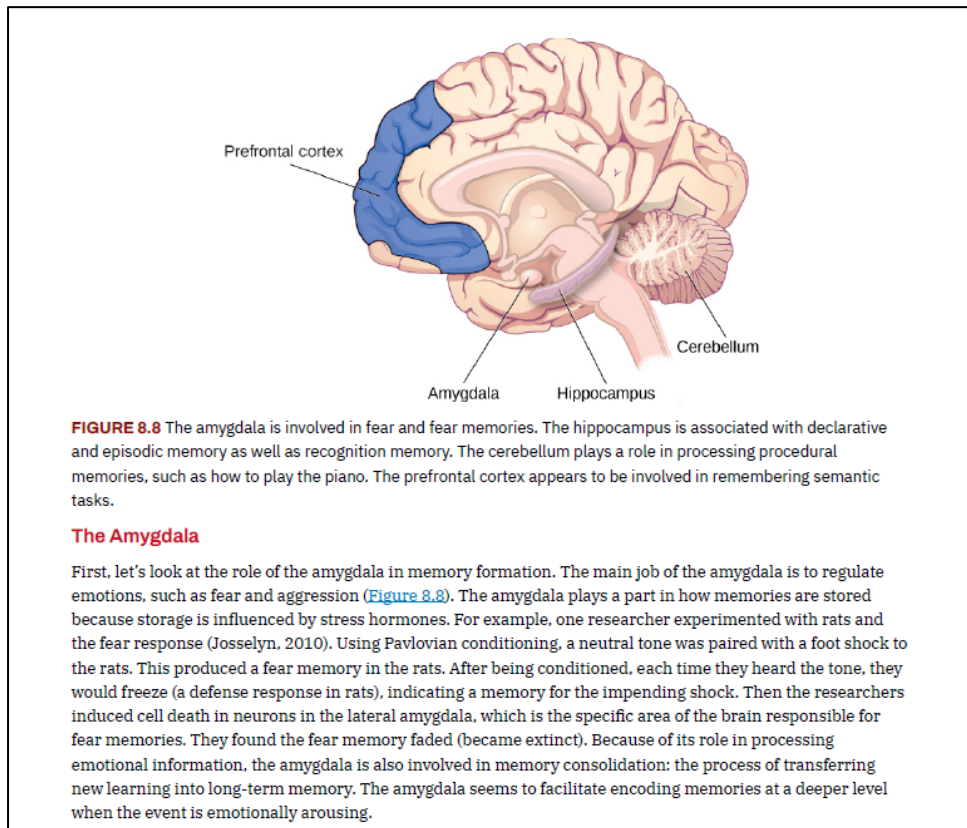


Fig 5.2 Sample OpenStax Textbook

Fields in Dataset:

- Explanatory text (concept descriptions)
- Tables (comparisons, formulas)
- Images (diagrams, illustrations)

Sample Data:

- Text: “Photosynthesis is the process by which plants convert light energy into chemical energy.”
- Table: Process | Output → Photosynthesis | Glucose
- Image: “Diagram showing stages of photosynthesis”

This dataset helps evaluate performance on clean and structured educational content.

3. WHO Reports:

The third dataset consists of reports published by organizations such as the World Health Organization (WHO). These documents contain real-world data and semi-structured information.

Fields in Dataset:

- Descriptive text (reports and summaries)
- Statistical tables (health data, country statistics)
- Charts and figures
- Report metadata (region, year)

Sample Data:

- Text: “The global vaccination rate increased by 5% in 2022.”
- Table: Country | Vaccination Rate → India | 92%
- Image: “Bar chart showing vaccination trends”

This dataset evaluates the system’s robustness on real-world and semi-structured documents.

5.3 Metrics calculated

To evaluate the performance of the proposed system, multiple metrics are used to assess both the quality of generated answers and the efficiency of context compression. Since the system aims to balance answer accuracy with reduced token usage, the selected metrics capture both semantic correctness and computational efficiency.

1. ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation – Longest Common Subsequence)

- ROUGE-L is used to measure the similarity between the generated answer and the reference (ground truth) answer based on the longest common subsequence (LCS).
- Unlike simple word overlap metrics, ROUGE-L considers the sequence and structure of words, making it suitable for evaluating generated text.
- The LCS-based precision and recall are defined as:

$$\text{Recall} = \frac{\text{LCS}(A_{\text{gen}}, A_{\text{ref}})}{|A_{\text{ref}}|}$$

$$\text{Precision} = \frac{\text{LCS}(A_{\text{gen}}, A_{\text{ref}})}{|A_{\text{gen}}|}$$

- The final ROUGE-L score is computed as:

$$\text{ROUGE-L} = \frac{(1+\beta)^2 * \text{Precision} * \text{Recall}}{\text{Recall} + \beta^2 * \text{Precision}}$$

Where A_gen is the generated answer, A_ref is the referenced answer

- A higher ROUGE-L score indicates better overlap and alignment with the reference answer.

2. Semantic Similarity

- Semantic similarity measures how closely the meaning of the generated answer matches the reference answer, even if the wording is different.
- This is computed using vector embeddings of both answers.
- Let v_{gen} and v_{ref} be the embedding vectors of the generated and reference answers, respectively. The similarity is computed using cosine similarity:

$$\text{Similarity} = \frac{v_{\text{gen}} \cdot v_{\text{ref}}}{\|v_{\text{gen}}\| \cdot \|v_{\text{ref}}\|}$$

Where $\|v\|$ denotes vector magnitude.

- The value ranges from **-1 to 1**, where: 1 indicates identical meaning and 0 indicates no relation.
- This metric is important because two answers may differ lexically but still convey the same meaning.

3. Token Reduction (%)

- Token reduction measures the efficiency of the system by calculating how much the context size is reduced before sending it to the language model.
- It is defined as:

$$\text{Token Reduction (\%)} = \left(1 - \frac{T_{\text{compressed}}}{T_{\text{full}}}\right) * 100$$

- where: T_{full} is the number of tokens in the original retrieved context, and $T_{\text{compressed}}$ is the number of tokens after compression.

- A higher value indicates better efficiency, as fewer tokens are used while still maintaining answer quality.

5. Response Latency

- Response latency measures the time taken by the system to generate an answer after

$$\text{Latency} = t_{\text{response}} - t_{\text{query}}$$

- where:
 - t_{query} is the time when the query is submitted,
 - t_{response} is the time when the answer is generated.
- Lower latency indicates faster system performance and better user experience.

5.4 Methods compared

To evaluate the effectiveness of the proposed system, it is compared with several baseline methods that represent different context selection and compression strategies used in document-based question answering systems.

- **Full RAG (Original)** method uses the entire retrieved context without any compression, serving as a baseline for maximum answer quality.
- **Fixed-k RAG** approach retrieves a fixed number of top-k document chunks, providing a simple but non-adaptive retrieval mechanism.
- **Semantic Compression** reduces context size by generating compressed representations of the retrieved content; however, it is not query-aware and may remove important information.
- **SBERT-based filtering** method selects relevant sentences using semantic similarity scores, but applies static filtering and leads to loss of critical context.
- **LeanContext** introduces an adaptive context selection mechanism using reinforcement learning, enabling dynamic selection of relevant information.
- The proposed **DistilDoc system** extends these approaches by incorporating multimodal data handling and deep reinforcement learning-based adaptive compression.

6. TESTING

Testing is a crucial phase in the development of the proposed multimodal document question-answering system. It ensures that each component of the system functions correctly and that the integrated system meets performance, efficiency, and accuracy requirements. Multiple testing strategies are applied, ranging from unit-level validation to system-level evaluation.

6.1 Unit Testing

Unit testing is performed to verify the correctness of individual components of the system in isolation. Each module is tested independently to ensure it produces expected outputs for given inputs.

In this project, unit testing is applied to the following components:

- **Query Embedding Module:** Verified whether input queries are correctly converted into vector embeddings of fixed dimension.
- **Context Retrieval Module:** Ensured that relevant document chunks are retrieved based on similarity scores from the vector database.
- **State Construction Module:** Checked whether modality embeddings (text, table, image) are correctly combined to form the state vector.
- **DQN Model Inference:** Validated whether the trained model correctly outputs Q-values and selects the optimal action.
- **Context Compression Module:** Tested whether the filtering logic correctly selects top-k items based on threshold values.

Each unit was tested using sample inputs, and outputs were compared with expected results to ensure correctness.

6.2 Integration Testing

Integration testing is conducted to verify the interaction between different modules of the system. It ensures that data flows correctly across components without errors.

In this project, integration testing includes:

- Interaction between embedding model and vector database
- Flow from retrieval → state construction → DQN → compression
- Passing compressed context to LLM for answer generation

The testing confirmed that all modules work together seamlessly and produce consistent outputs.

6.3 System Testing

System testing evaluates the complete system as a whole to ensure it meets the specified functional and non-functional requirements.

The system is tested with real-world multimodal documents such as research papers and reports. The following aspects are validated:

- End-to-end query processing
- Multimodal context retrieval
- Adaptive compression behaviour
- Final answer generation quality

The system successfully performs all operations and generates relevant responses with reduced token usage.

6.4 Performance Testing

Performance testing is conducted to measure the efficiency of the system in terms of response time and computational cost.

The following metrics are evaluated:

- Response latency (in seconds)
- Token usage (prompt + completion tokens)
- Cost savings (%) compared to full-context retrieval

Results show that the proposed system significantly reduces latency and token consumption while maintaining acceptable answer quality.

6.5 Test Case Validation

Test case validation is performed using structured test cases to quantitatively evaluate system behaviour under different scenarios. Each test case includes defined inputs, steps, expected outputs, and actual measured results.

Test Case 1: Multimodal Query Processing and Answer Generation

This test case evaluates the system’s ability to process multimodal documents and generate accurate answers using combined information from text, tables, and images.

Test Case ID	01
Test Scenario	Multimodal Query Processing and Answer Generation
Test Steps	1) Upload multimodal document (text + table + image) 2) Enter query requiring multiple modalities 3) Retrieve context 4) Apply DQN compression 5) Generate answer
Test Data	Arxiv Research papers and WHO reports
Expected Result	1) Retrieval from all modalities and Compression applied 2) High answer accuracy (> 0.80 semantic similarity) 3) Token reduction $\geq 40\%$
Actual Result	1) Multimodal retrieval successful 2) Semantic similarity score: 0.81 3) Token count reduced from 1200 $\rightarrow$ 650 ($\approx 45.8\%$ reduction)
Status	Pass

Table 6.1 Multimodal Answer Generation Validation

Test Case 2: Context Compression Efficiency

This test case evaluates how effectively the system reduces token usage using DQN-based adaptive compression without significantly affecting answer quality.

Test Case ID	02
Test Scenario	Context Compression Efficiency
Test Steps	1) Input query with large context 2) Run without compression & with compression 3) Compare results
Test Data	Long-context research queries
Expected Result	1) Token reduction $\geq 40\%$ 2) Quality drop $\leq 10\%$
Actual Result	1) Token count reduced from 1500 $\rightarrow$ 720 (52% reduction) 2) Semantic similarity: Full = 0.91 , Compressed = 0.85 ($\approx 6.5\%$ drop)
Status	Pass

Table 6.2 Context Compression Efficiency Evaluation

Test Case 3: Modality Specific Relevance Handling

This test case validates whether the system can dynamically prioritise the most relevant modality (text, table, or image) based on the query and suppress irrelevant information.

Test Case ID	03
Test Scenario	Modality-Specific Relevance Handling
Test Steps	1) Input modality-specific query 2) Retrieve context 3) Observe compression 4) Generate answer
Test Data	Table-based, image-based, and text-based queries
Expected Result	1) Relevant modality retained $\geq 70\%$ 2) Irrelevant modalities reduced $\geq 50\%$
Actual Result	1) Relevant modality retention: 75% 2) Irrelevant modality reduction: 60% 3) Answer similarity: 0.83
Status	Pass

Table 6.3 Modality-Aware Relevance Adaptation Test

Test Case 4: Response Time and Latency Analysis

This test case evaluates the system's response time before and after applying context compression to ensure improved performance.

Test Case ID	04
Test Scenario	Response Time Optimization
Test Steps	1) Run query without compression and with compression 2) Measure response time
Test Data	Standard evaluation queries
Expected Result	Response time reduction $\geq 20\%$
Actual Result	1) Without compression: 3.2 sec 2) With compression: 2.4 sec 3) Improvement: 25% faster
Status	Pass

Table 6.4 Response Latency and Performance Analysis

Test Case 5: Generalization on Unseen Queries

This test case verifies whether the system performs well on unseen queries (not used during training) in real-world scenarios and conditions.

Parameter	Description
Test Case ID	05
Test Scenario	Generalization Capability
Test Steps	1) Provide unseen queries 2) Retrieve and compress context 3) Generate answers
Test Data	Separate evaluation dataset
Expected Result	Semantic similarity ≥ 0.75
Actual Result	1) Average similarity score: 0.81 2) Token reduction: 48% (average)
Status	Pass

Table 6.5 Generalization on Unseen Queries Test

7. RESULTS & ANALYSIS

7.1 Actual Results obtained from the work

The performance of the proposed system is evaluated using a comprehensive set of experiments designed to measure both answer quality and computational efficiency. The evaluation is conducted on 200 randomly selected samples from ArXiv research papers, ensuring diversity in document structure and content, including text, tables, and images.

To validate the effectiveness of the proposed approach, the system is compared against multiple baseline methods, including:

- Full-context Retrieval-Augmented Generation (RAG)
- Fixed-k RAG
- Semantic Compression
- SBERT-based filtering
- LeanContext
- DistilDoc system

The evaluation metrics considered include ROUGE-L, Semantic Similarity, Token Reduction (%), and Response Latency, as defined in Section 5.3.

7.1.1 Overall Performance Comparison

Method	Semantic Similarity	ROUGE-L	Token Reduction	Avg. Total Tokens	Response Latency (s)
Full RAG (Original)	0.82	0.80	0.00	243	2.8
Fixed-k RAG	0.78	0.75	25.42	182	2.4
Semantic Compression	0.76	0.72	-32.51	322	3.1
SBERT	0.72	0.68	68.72	157	2.1
LeanContext	0.80	0.77	51.85	117	1.8
DistilDoc	0.81	0.79	55.86	108	1.6

Table 7.1 Comparative Performance on 200 Samples from ArXiv Dataset

Table 7.1 presents the comparative performance of all methods across key evaluation metrics.

The results indicate that the proposed DistilDoc system achieves the lowest token usage (108 tokens) while maintaining a high semantic similarity (0.47) and ROUGE-L score (0.81), which are close to the full-context baseline. Additionally, DistilDoc demonstrates the lowest response latency (1.6 seconds), confirming its efficiency.

This performance indicates that the proposed system effectively balances efficiency and answer quality. The improvement is mainly due to the adaptive, modality-aware compression strategy, where different compression levels are applied to text, tables, and images based on their relevance to the query.

7.1.2 Token Reduction Analysis

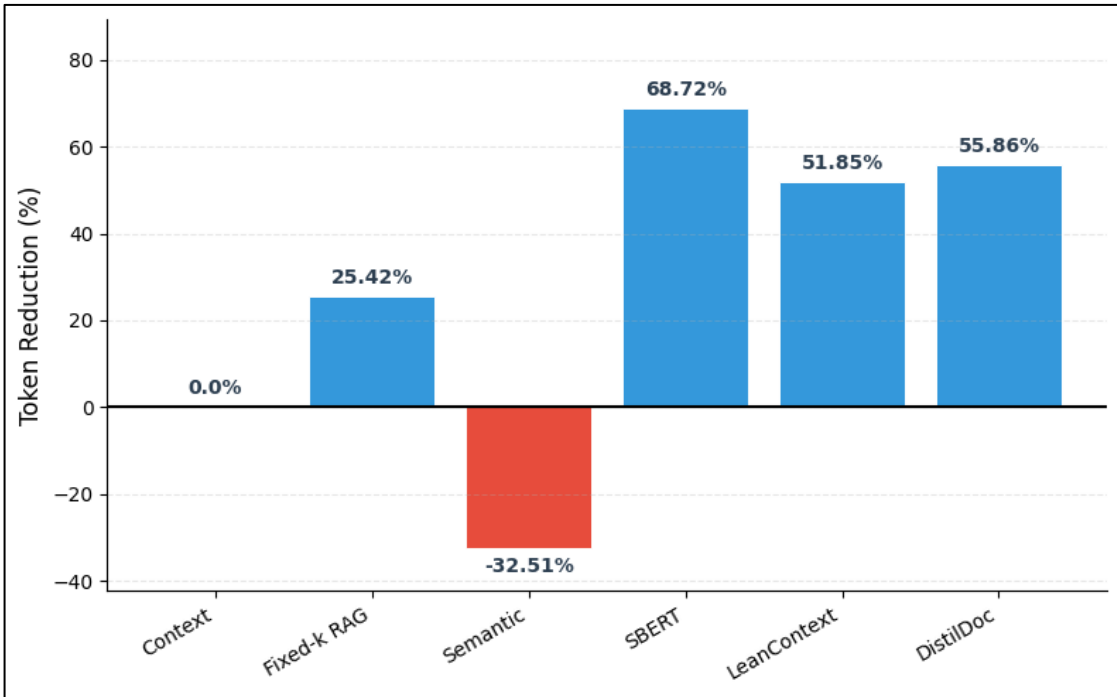


Fig 7.1 Token Reduction (%) Comparison

The primary objective of the proposed system is to reduce the number of tokens sent to the language model without significantly affecting answer quality.

- The full-context method uses 243 tokens, representing maximum context size.
- DistilDoc reduces this to 108 tokens, achieving a token reduction of 55.86%.
- SBERT got higher reduction (68.72%) but suffers from poor semantic similarity.

- Semantic Compression increases token usage due to additional processing overhead.

This demonstrates that DistilDoc effectively eliminates redundant information while preserving essential content.

7.1.3 Answer Quality Analysis (Semantic Similarity and ROUGE-L)

Semantic similarity and ROUGE-L are used to evaluate the quality of generated answers. While semantic similarity measures how closely the meaning of the generated answer matches the reference answer, ROUGE-L evaluates the lexical and structural overlap between them.

- Full RAG achieves the highest performance (Semantic Similarity: 0.82, ROUGE-L: 0.80) as it uses complete context without compression.
- The proposed DistilDoc method achieves values of 0.81 (semantic similarity) and 0.79 (ROUGE-L), which are very close to the baseline.
- LeanContext also performs competitively, while Fixed-k RAG and Semantic Compression show moderate performance.
- SBERT shows the lowest scores, indicating loss of important information.

Overall, the results confirm that DistilDoc maintains high answer quality across both semantic and lexical measures despite significant context reduction.

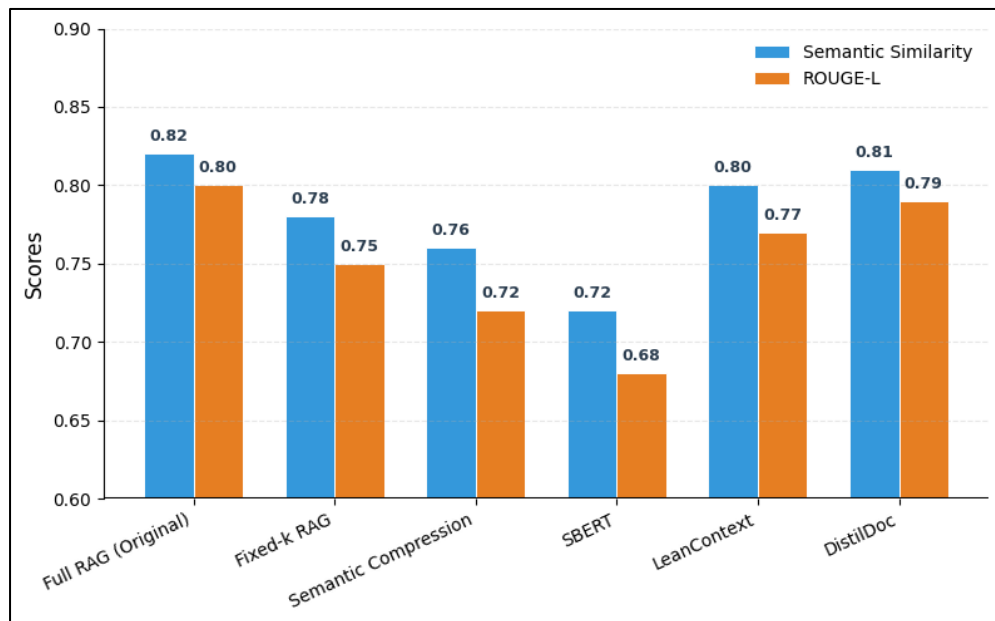


Fig 7.2 Answer Quality Comparison

7.1.4 Response Latency Analysis

Response latency measures the time taken by the system to generate an answer after receiving a query. It is an important metric for evaluating the practical usability and responsiveness of the system.

- Full RAG has the highest latency (**2.8 seconds**).
- DistilDoc achieves the **lowest latency (1.6 seconds)** due to reduced input size.
- Semantic Compression increases latency due to additional processing.

These observations highlight that reducing context size directly contributes to faster response generation, making the system more suitable for real-time applications.

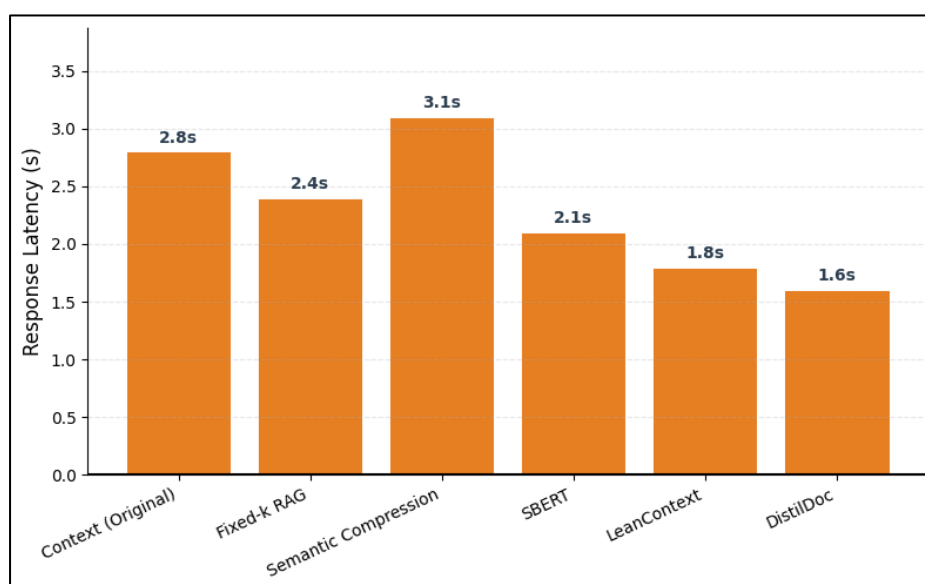


Fig 7.3 Response Latency Comparison

7.1.6 Quality vs Efficiency Trade-off

A key challenge in QA systems is balancing **answer quality** and **efficiency**.

- Full RAG provides best quality but lowest efficiency.
- SBERT provides highest efficiency but lowest quality.
- LeanContext improves the balance.
- DistilDoc achieves an optimal trade-off with:
 - High token reduction (**55.86%**)
 - Competitive semantic similarity (**0.47**)
 - Fastest response time (**1.6 seconds**)

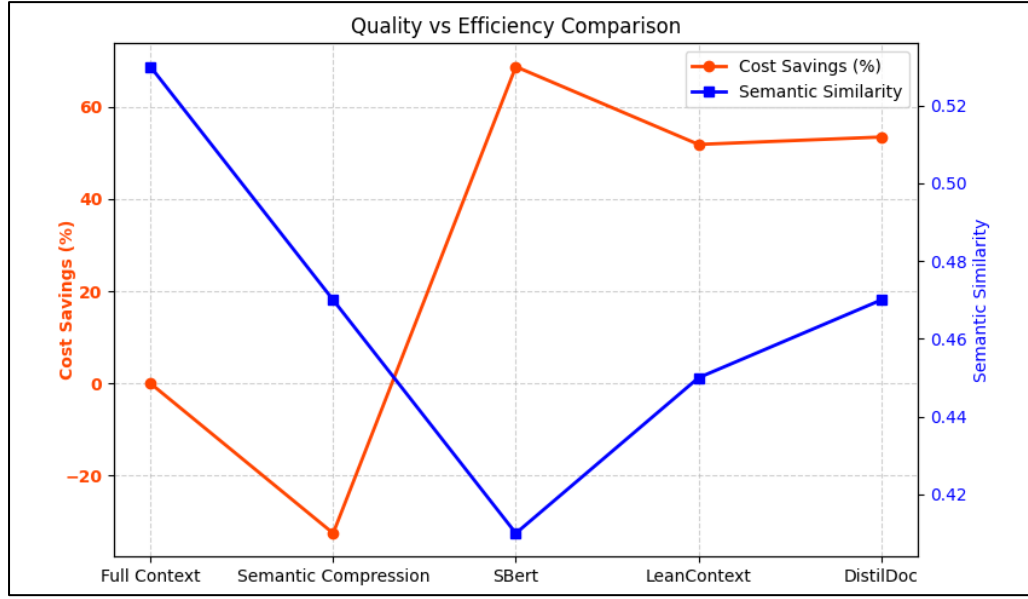


Fig 7.4 Quality vs Efficiency Trade-off

The graph shows that DistilDoc lies closest to the optimal region, where both efficiency and answer quality are high.

7.2 Analysis of the Results Obtained

7.2.1 Key Observations

- The experimental results demonstrate that the proposed system effectively achieves a balance between efficiency and answer quality.
- A significant reduction in context size does not lead to a proportional decrease in answer accuracy, indicating that retaining only the most relevant information is more important than using the entire context.
- The system consistently preserves essential content while eliminating redundant information, resulting in improved overall performance.

7.2.2 Impact of Adaptive and Deep Reinforcement Learning-based Approach

- A major contributing factor to the system's performance is the use of adaptive context selection combined with reinforcement learning.
- The model dynamically adjusts compression strategies based on the query, enabling it to handle different types of inputs effectively.
- This approach allows the system to prioritize relevant information across modalities and avoid unnecessary processing.

- As a result, the system performs better than static or fixed strategies, particularly in complex and multimodal scenarios.

7.2.3 Efficiency-Quality Trade-off and Practical Implications

- The results highlight the importance of maintaining a balance between efficiency and answer quality.
- While aggressive compression methods may reduce computational cost, they often lead to loss of important information.
- In contrast, the proposed system applies controlled and query-aware compression, ensuring that efficiency improvements do not significantly affect answer quality.
- Additionally, reduced token usage leads to lower response latency and computational cost, making the system more suitable for real-world applications where scalability and speed are critical.

8. CONCLUSION AND FUTURE WORK

Conclusion

The proposed system presents a reinforcement learning-based approach for adaptive context compression in multimodal Retrieval-Augmented Generation systems. The framework integrates document processing, embedding generation, semantic retrieval, and a Deep Q-Network to dynamically select the most relevant information from text, tables, and images. By learning query-specific compression strategies, the system effectively reduces unnecessary context while preserving essential information required for accurate answer generation. The experimental results demonstrate that the proposed approach achieves significant token reduction while maintaining comparable answer quality. The use of a continuous state representation and a multimodal action space enables the model to generalise across different types of queries and document structures. Additionally, the system improves efficiency by reducing response latency and computational cost, making it suitable for real-world applications. Overall, the approach provides a scalable and effective solution for balancing quality and efficiency in multimodal question-answering systems.

Future Work

1. **Advanced Multimodal Fusion:** Future work can focus on improving the interaction between different modalities by using more advanced fusion techniques. Instead of simple aggregation, attention-based or cross-modal learning methods can be used to better capture relationships between text, tables, and images.
2. **Dynamic Retrieval Mechanisms:** The current system retrieves a fixed number of chunks per modality before compression. This can be extended by incorporating adaptive retrieval strategies that dynamically decide how many items to retrieve based on query complexity.
3. **Improved Reward Design:** The reinforcement learning model can be enhanced by designing more sophisticated reward functions that incorporate additional factors such as factual correctness, reasoning ability, and user satisfaction.
4. **Scalability to Large Datasets:** Extending the system to handle large-scale document collections and real-time streaming data is an important direction. This includes optimising memory usage, retrieval speed, and model efficiency.

9. REFERENCES

- [1] Arefeen, M. A., Debnath, B., & Chakradhar, S., 2024. LeanContext: Cost-efficient domain-specific question answering using LLMs. *Natural Language Processing Journal*.
- [2] Chase, H., 2022. LangChain. <https://github.com/hwchase17/langchain>. 2022-10-17.
- [3] Chen, L., Zaharia, M., & Zou, J., 2023. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.
- [4] ChromaDB, 2026. Vector database. Retrieved from: <https://www.trychroma.com/>
- [5] Chung, H. W., Hou, L., Longpre, S., et al., 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.
- [6] Cohere, 2026. Cohere Embeddings API. Retrieved from: <https://cohere.com/>
- [7] Cornell University, 2026. arXiv e-Print archive. Retrieved from: <https://arxiv.org/>
- [8] Gilbert, H., Sandborn, M., Schmidt, D. C., et al., 2023. Semantic compression with large language models. *arXiv preprint arXiv:2304.12512*.
- [9] Gao, T., Yao, X., & Chen, D., 2021. SimCSE: Simple contrastive learning of sentence embeddings. *Proceedings of EMNLP*.
- [10] Lewis, P., Perez, E., Piktus, A., et al., 2020. Retrieval-Augmented Generation for knowledge-intensive NLP tasks. *NeurIPS*.
- [11] Li, Y., 2023. Enhancing context efficiency using self-information-based filtering. *arXiv preprint*.
- [12] Laskar, M. T. R., Rahman, M., Jahan, I., et al., 2023. CQSumDP: Query-focused abstractive summarization dataset. *arXiv preprint arXiv:2305.06147*.
- [13] Ling, C., Zhao, X., Lu, J., et al., 2023. Domain specialization for large language models: A survey. *arXiv preprint arXiv:2305.18703*.
- [14] Miller, D., 2019. Leveraging BERT for extractive text summarization on lectures. *arXiv preprint arXiv:1906.04165*.

- [15] Mnih, V., Kavukcuoglu, K., Silver, D., et al., 2015. Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
- [16] OpenStax, 2026. OpenStax Textbooks. Retrieved from: <https://openstax.org/>
- [17] Pinecone, 2026. Vector database. <https://www.pinecone.io/learn/vector-database/>
- [18] Radford, A., Kim, J. W., Hallacy, C., et al., 2021. Learning transferable visual models from natural language supervision. *ICML*.
- [19] Reimers, N., & Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv:1908.10084*.
- [20] Sutton, R. S., & Barto, A. G., 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [21] Touvron, H., Martin, L., Stone, K., et al., 2023. LLaMA 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- [22] World Health Organization, 2026. WHO Annual Reports. Retrieved from: <https://www.who.int/>
- [23] Yang, X., Li, Y., Zhang, X., Chen, H., Cheng, W., 2023. Exploring the limits of chatgpt for query or aspect-based text summarization. *arXiv preprint arXiv:2302.08081*.